ESCUELA DE INGENIERÍA DE FUENLABRADA

GRADO EN INGENIERÍA TELEMÁTICA

TRABAJO FIN DE GRADO

Versión Web Interactiva del Juego Social *Verdad o Reto*

Autor: Jose Luis Tamayo Díez

Tutor: Miguel Ángel Ortuño Pérez

Curso académico 2025/2026
Universidad Rey Juan Carlos

© 2026 Jose Luis Tamayo Díez.

Esta obra está bajo la licencia
Creative Commons Atribución–CompartirIgual 4.0 Internacional (CC BY-SA 4.0),
disponible en:
`https://creativecommons.org/licenses/by-sa/4.0/deed.es`

El código fuente del proyecto se encuentra disponible públicamente en:
`https://github.com/jltamayo2018/tfg-verdad-reto`
bajo licencia MIT.

Resumen

Este proyecto presenta el diseño y desarrollo de una versión web interactiva del juego social *Verdad o Reto*, adaptada a un entorno digital moderno con funcionalidades avanzadas de colaboración y juego en línea. Como antecedentes, se parte de la popularidad de este tipo de dinámicas lúdicas en contextos sociales presenciales y de la creciente demanda de plataformas web capaces de ofrecer estas experiencias a través de Internet.

El clásico juego *Verdad o Reto* consiste en una actividad grupal donde los participantes deberán responder a una pregunta personal o realizar un desafío por turnos.

El objetivo principal del trabajo ha sido diseñar e implementar una plataforma web que permita a los usuarios gestionar y compartir sus propios *packs* de preguntas y retos, así como participar en partidas tanto locales como a distancia mediante salas online sincronizadas en tiempo real.

Para su desarrollo, se ha empleado el *framework Django* como base para el *backend*, *PostgreSQL* como sistema gestor de bases de datos y *Django Channels* para la comunicación en tiempo real mediante *WebSockets*. La interfaz web se ha implementado mediante plantillas *HTML*, estilos *CSS* y *JavaScript*. Asimismo, para habilitar las salas de videoconferencia en partidas remotas, se ha integrado la herramienta externa *Jitsi*. El despliegue se ha realizado mediante la plataforma *Render*, configurando un entorno de producción accesible públicamente.

Como resultado, se ha obtenido una aplicación web funcional, desplegada y operativa, que permite la gestión estructurada de contenido colaborativo, la creación de partidas *online* y la interacción simultánea de múltiples usuarios. Las pruebas realizadas muestran un funcionamiento estable bajo condiciones normales de uso.

En conclusión, el proyecto demuestra la viabilidad técnica de trasladar una dinámica social tradicional a un entorno web interactivo, combinando tecnologías modernas de desarrollo y despliegue en la nube, y sienta las bases para posibles ampliaciones futuras en términos de escalabilidad y nuevas funcionalidades.

Abstract

This project presents the design and development of an interactive web-based version of the social game *Truth or Dare*, adapted to a modern digital environment with advanced collaboration and online gameplay features. The initiative is motivated by the popularity of this type of social activity in face-to-face settings and the growing demand for web platforms capable of offering these experiences through the Internet.

The classic *Truth or Dare* game consists of a group activity in which participants take turns answering a personal question or completing a challenge.

The main objective of this work is to design and implement a web platform that allows users to manage and share their own *packs* of questions and challenges, as well as participate in both local and remote games through synchronized online rooms in real time.

The system has been developed using the *Django* framework as the backend foundation, *PostgreSQL* as the database management system, and *Django Channels* to enable real-time communication through *WebSockets*. The web interface has been implemented through *HTML* templates, *CSS* styles and *JavaScript*. Additionally, the external tool *Jitsi* has been integrated to provide videoconferencing capabilities during remote matches. The application has been deployed on the *Render* platform, configuring a publicly accessible production environment.

As a result, a functional and fully deployed web application has been obtained, enabling structured management of collaborative content, the creation of online game sessions and the simultaneous interaction of multiple users. The tests carried out show stable performance under normal usage conditions.

In conclusion, the project demonstrates the technical feasibility of adapting a traditional social activity to an interactive web environment by combining modern development technologies with cloud deployment solutions. Furthermore, it establishes a foundation for future improvements in terms of scalability and additional features.

Índice general

1. Introducción **1**
1.1. Contexto y planteamiento del problema 1
1.2. Motivación y objetivos del proyecto 1
1.3. Alcance y público objetivo 2
1.4. Estructura de la memoria 3

2. Estado del arte y tecnologías **4**
2.1. *Python* y *Django* 4
2.1.1. *Python* 4
2.1.2. *Django* 5
2.2. Patrón *MVT* 5
2.3. Tecnologías web 6
2.3.1. *HTML* 6
2.3.2. *CSS* 7
2.3.3. Diseño *responsive* 7
2.3.4. Interactividad y usabilidad 7
2.4. Autenticación, autorización y seguridad 8
2.4.1. Gestión de usuarios 8
2.4.2. Protección *CSRF* y control de acceso 8
2.5. Persistencia de datos 9
2.5.1. *SQLite* 9
2.5.2. *PostgreSQL* 10
2.6. Acceso público mediante enlaces y códigos QR 10
2.6.1. Generación y uso de enlaces públicos 10
2.6.2. Generación de códigos QR 10
2.7. Comunicación en tiempo real y salas *online* 11
2.7.1. Comunicación en tiempo real en aplicaciones web 11
2.7.2. *Jitsi Meet* 11
2.8. Opciones de despliegue y comparativa de plataformas 12

3. Análisis y diseño **13**
3.1. Casos de uso . . . 13
3.1.1. Actores . . . 13
3.1.2. Supuestos y límites . . . 14
3.1.3. Diagrama de casos de uso . . . 14
3.1.4. Descripción de los casos de uso . . . 15
3.2. Modelo de datos . . . 17
3.2.1. Entidades principales . . . 17
3.2.2. Diagrama entidad-relación . . . 18
3.2.3. Campos y relaciones . . . 19
3.3. Arquitectura del sistema . . . 21
3.3.1. Visión general . . . 21
3.3.2. Diagrama de arquitectura . . . 21
3.3.3. Capas del sistema . . . 23
3.3.4. Flujo de información . . . 24
3.3.5. Integración de sistemas internos . . . 25
3.4. Diseño de la interfaz y navegación . . . 26
3.4.1. Estructura de navegación . . . 26
3.4.2. Plantillas principales . . . 27
3.4.3. Diseño visual y estilo . . . 27
3.4.4. Adaptabilidad y *responsive design* . . . 28
3.5. Usabilidad y accesibilidad . . . 28
3.5.1. Principios de usabilidad . . . 28
3.5.2. Criterios de accesibilidad . . . 29
3.5.3. Validación de accesibilidad . . . 29

4. Implementación **31**
4.1. Estructura del proyecto *Django* . . . 31
4.2. Gestión de usuarios y autenticación . . . 33
4.2.1. Sistema de autenticación base . . . 33
4.2.2. Implementación del sistema de recuperación de contraseña . . . 34
4.3. Implementación de *packs* y acciones . . . 36
4.4. Sistema de colaboración y permisos . . . 38
4.5. Generación de enlaces públicos y códigos QR . . . 39
4.6. Vista pública del juego . . . 40
4.7. Implementación de salas *online* . . . 41
4.7.1. Modelo de datos de la sala . . . 41
4.7.2. Gestión del ciclo de vida de la sala . . . 42
4.7.3. Comunicación en tiempo real (*Django Channels*) . . . 43

4.7.4. Integración con *Jitsi* 43
4.8. Gestión de errores y *logging* 45

5. Integración y despliegue **46**
5.1. Entorno de desarrollo 46
5.2. Configuración de producción 47
5.3. Archivos estáticos y `collectstatic` 49
5.4. Base de datos en producción 50
5.5. Despliegue en la nube 51
5.5.1. *Render* 51
5.5.2. Proceso de despliegue 51
5.6. Dominio, *DNS* y HTTPS 52
5.7. Monitorización y mantenimiento 52

6. Pruebas y validación **54**
6.1. Plan de pruebas 54
6.2. Pruebas funcionales 55
6.3. Pruebas de integración 58
6.4. Pruebas de usabilidad 60
6.5. Evaluación de rendimiento 61

7. Resultados y discusión **65**
7.1. Cumplimiento de objetivos 65
7.2. Limitaciones 66
7.3. Lecciones aprendidas 67

8. Conclusiones y trabajos futuros **68**
8.1. Conclusiones 68
8.2. Líneas de trabajo futuro 69

A. Manual de usuario **73**
A.1. Introducción 73
A.2. Acceso a la aplicación 73
A.3. Registro e inicio de sesión 74
A.3.1. Registro de usuario 75
A.3.2. Inicio de sesión 75
A.3.3. Recuperación de contraseña 76
A.4. Panel de usuario: Mis packs 77
A.4.1. Creación de un nuevo pack 78
A.5. Edición y gestión de un pack 79
A.5.1. Información general del pack 79

A.5.2. Colaboradores 80
A.5.3. Gestión de preguntas y retos 81
A.5.4. Compartir y jugar 82
A.6. Vista pública del *pack* 83
A.7. Salas *online* 83
A.8. *Packs* compartidos 84

Índice de figuras

3.1. Diagrama de casos de uso del sistema. 15
3.2. Diagrama entidad–relación del sistema. 19
3.3. Diagrama de arquitectura general del sistema. 22

4.1. Estructura general del proyecto *Django*. 32

A.1. Página principal de la aplicación. 74
A.2. Formulario de registro. 75
A.3. Formulario de inicio de sesión. 76
A.4. Formulario de recuperación de contraseña. 77
A.5. Vista de gestión de packs personales del usuario. 78
A.6. Formulario de creación de un nuevo *pack*. 79
A.7. Información general del pack. 80
A.8. Gestión de colaboradores y permisos del pack. 80
A.9. Gestión de preguntas y retos del pack. 81
A.10. Opciones de acceso, compartición y juego del pack. 82
A.11. Vista pública del *pack*. 83
A.12. Interfaz de una sala online con videocomunicación integrada. 84
A.13. Listado de packs compartidos con el usuario. 85

Índice de tablas

5.1. Comparativa entre entorno de desarrollo y entorno de producción. 48

6.1. Resumen de pruebas funcionales realizadas 57
6.2. Tiempos medios observados en entorno de producción. 63

Capítulo 1

Introducción

El objetivo de este capítulo es situar al lector en el contexto del proyecto, explicando el problema que da origen a la propuesta y las razones que motivan su desarrollo. Finalmente, se detalla el alcance del trabajo y la estructura general de la memoria.

1.1. Contexto y planteamiento del problema

En la actualidad, las aplicaciones web interactivas desempeñan un papel relevante en el ámbito del entretenimiento, la educación y la comunicación social. Su facilidad de acceso desde distintos dispositivos y la posibilidad de ofrecer experiencias compartidas en tiempo real las hace especialmente atractivas en entornos sociales y recreativos.

El clásico juego de *Verdad o Reto* es una dinámica popular en reuniones, fiestas o encuentros informales. En esta actividad, los participantes deben elegir entre responder sinceramente a una pregunta personal o realizar un desafío propuesto por el grupo. Su carácter espontáneo y participativo lo convierte en una actividad lúdica de fácil integración en distintos contextos sociales.

Sin embargo, muchas de las versiones disponibles en línea presentan diversas limitaciones. Entre ellas se encuentran la falta de personalización de preguntas y retos, la presencia de publicidad invasiva, la necesidad de suscripción para acceder a funcionalidades avanzadas o una adaptación deficiente a determinados tamaños de pantalla.

Estas carencias evidencian la oportunidad de desarrollar una solución web que permita modernizar esta dinámica tradicional, ofreciendo mayor flexibilidad, accesibilidad y control sobre el contenido del juego.

1.2. Motivación y objetivos del proyecto

La motivación principal de este proyecto surge del interés por combinar el aprendizaje práctico de desarrollo web con la creación de una aplicación interactiva completa, abarcando todas las

fases del proyecto: desde su concepción inicial hasta su despliegue en producción.

Desde el punto de vista académico, el proyecto permite aplicar de forma integrada los conocimientos adquiridos durante el Grado en Ingeniería, incluyendo programación web, diseño de interfaces, modelado de bases de datos, principios de seguridad y despliegue de aplicaciones en entornos reales. Asimismo, representa una oportunidad para desarrollar una solución funcional orientada a la experiencia de usuario.

Objetivo general

Desarrollar una plataforma web interactiva que permita jugar a *Verdad o Reto* de forma accesible, personalizable y adaptable a distintos dispositivos, superando las limitaciones identificadas en las versiones existentes.

Objetivos específicos

- Diseñar una interfaz clara, intuitiva y adaptable a distintos tamaños de pantalla.
- Permitir la creación y gestión de *packs* personalizados de preguntas y retos.
- Implementar un sistema de acceso público mediante enlaces y códigos QR.
- Incorporar funcionalidades de interacción en tiempo real mediante salas *online*.
- Garantizar un nivel adecuado de seguridad en la gestión de usuarios y datos.
- Desplegar la aplicación en un entorno de producción accesible desde Internet.

1.3. Alcance y público objetivo

La aplicación está diseñada para ser utilizada por grupos de amigos, estudiantes o familias que busquen una forma rápida y divertida de entretenerse, tanto de manera presencial desde un único dispositivo como de forma remota a través de Internet. De esta forma, la aplicación permite escenarios de juego remoto, complementando el uso presencial tradicional.

La interfaz de la aplicación es sencilla e intuitiva, y su diseño *responsive* permite que funcione correctamente en dispositivos móviles, *tablets* y ordenadores, garantizando una experiencia de uso coherente en distintos tamaños de pantalla.

El sistema incluye un panel de administración desde el que los usuarios autenticados pueden crear y editar *packs* de preguntas y retos, gestionando su contenido de forma totalmente personalizada y adaptándolo a distintos contextos de uso.

En cuanto al alcance técnico, el proyecto abarca el desarrollo completo del *frontend*, el *backend*, así como el despliegue de la aplicación en un servidor público. Esto incluye la gestión persistente de los datos, la implementación de mecanismos de seguridad y control de acceso, y una presentación visual coherente con la identidad del proyecto.

1.4. Estructura de la memoria

La presente memoria se estructura en varios capítulos que recogen de forma ordenada todo el proceso de desarrollo del proyecto:

- **Capítulo 1: Introducción.** Presenta el contexto en el que surge el proyecto, los objetivos principales planteados y la motivación que impulsó su desarrollo, así como la estructura general de la memoria.
- **Capítulo 2: Estado del arte y tecnologías.** Describe las herramientas y lenguajes empleados, tanto en el *backend* como en el *frontend*, incluyendo los mecanismos de seguridad, la persistencia de datos, el acceso público mediante enlaces y códigos QR, la comunicación en tiempo real mediante salas *online* y las opciones de despliegue en la nube.
- **Capítulo 3: Análisis y diseño.** Expone la estructura del sistema, los casos de uso, el diseño del modelo de datos y las decisiones adoptadas con respecto a la interfaz, la navegación, la usabilidad y la accesibilidad.
- **Capítulo 4: Implementación.** Detalla la implementación de la aplicación web utilizando el *framework Django*, describiendo la estructura del proyecto y el desarrollo de las principales funcionalidades, como la gestión de *packs*, la colaboración entre usuarios, el acceso público al juego y la creación de salas *online*.
- **Capítulo 5: Integración y despliegue.** Explica el entorno de desarrollo utilizado y el proceso de configuración y despliegue de la aplicación en un entorno de producción, incluyendo la gestión de bases de datos, archivos estáticos, dominio y seguridad.
- **Capítulo 6: Pruebas y validación.** Describe las pruebas realizadas durante el desarrollo del proyecto, centradas principalmente en pruebas funcionales, de integración y de usabilidad, así como una evaluación básica del rendimiento del sistema.
- **Capítulo 7: Resultados y discusión.** Analiza el grado de cumplimiento de los objetivos propuestos, las limitaciones detectadas durante el desarrollo y las principales lecciones aprendidas a lo largo del proyecto.
- **Capítulo 8: Conclusiones y trabajos futuros.** Presenta las conclusiones finales del proyecto y plantea posibles líneas de trabajo futuro y mejoras que podrían abordarse en desarrollos posteriores.

Finalmente, la memoria se completa con un anexo que incluye el manual de usuario, en el que se describe de forma detallada el funcionamiento del sistema a través de los principales flujos de uso de la aplicación.

Capítulo 2

Estado del arte y tecnologías

En este capítulo se presenta el estado del arte y el conjunto de tecnologías empleadas en el desarrollo del proyecto. Se describen los principales conceptos, herramientas y enfoques técnicos que sustentan la implementación del sistema, así como su relevancia en el contexto actual del desarrollo web.

El análisis de estas tecnologías permite justificar las decisiones adoptadas durante el diseño e implementación de la aplicación, proporcionando un marco teórico y práctico sobre el que se construye la solución propuesta.

2.1. *Python* y *Django*

2.1.1. *Python*

En la actualidad existen numerosos lenguajes de programación que permiten la posibilidad de desarrollar aplicaciones web y de escritorio. Entre ellos, *Python*[1] destaca por su sencillez, legibilidad y potencia, convirtiéndose en uno de los lenguajes más populares y versátiles en la industria del *software*.

Python es un lenguaje de programación de alto nivel, normalmente interpretado, aunque también puede compilarse. Su sintaxis clara y su tipado dinámico facilitan la comprensión del código y reducen la complejidad del desarrollo y el mantenimiento de proyectos. Gracias a estas características, se ha consolidado como una herramienta de referencia en ámbitos académicos, científicos y empresariales.

Entre las cualidades que hacen de *Python* una herramienta adecuada para este proyecto destacan su sintaxis legible, que facilita el desarrollo y mantenimiento del código, y la gran cantidad de librerías disponibles para tareas como el desarrollo web, la gestión de datos o la automatización. Además, su compatibilidad con prácticamente cualquier sistema operativo utilizado en la actualidad, así como el respaldo de una comunidad activa que mantiene el lenguaje actualizado y bien documentado, lo convierten en una opción segura y eficiente para proyectos de cualquier tamaño.

En el contexto de este proyecto, *Python* se emplea principalmente como lenguaje base para el *framework Django*, encargado de gestionar la lógica del servidor, las peticiones *HTTPS*, la seguridad y la conexión con la base de datos. Su elección se debe a su equilibrio entre simplicidad, rendimiento y robustez, además de su idoneidad para el desarrollo rápido de aplicaciones web.

2.1.2. *Django*

Sobre la base del lenguaje *Python*, este proyecto utiliza el *framework Django*[2], una herramienta de *software* libre que facilita el desarrollo de aplicaciones web, diseñada para simplificar la creación de aplicaciones seguras, escalables y mantenibles. *Django* se fundamenta en el patrón de arquitectura *MVT (Model–View–Template)*, que separa claramente la lógica de negocio, la presentación y el acceso a datos, facilitando el mantenimiento y la evolución del sistema.

Asimismo, incorpora medidas de seguridad integradas frente a ataques comunes como *XSS*[3], *CSRF* o inyecciones *SQL*[4]. Además, genera automáticamente un panel de administración que facilita la gestión de los modelos de datos. A ello se suma su sistema de plantillas, que permite separar el diseño del contenido dinámico, simplificando el mantenimiento de la interfaz y garantizando una experiencia de usuario coherente.

En el contexto del proyecto *Verdad o Reto*, *Django* actúa como el núcleo del sistema, gestionando las vistas, los modelos y la autenticación de usuarios, y organizando la comunicación entre las distintas capas del patrón MVT. Esta estructura garantiza una base sólida, modular y fácil de ampliar. Además, *Django* simplifica el despliegue en entornos de producción gracias a su compatibilidad con servidores *ASGI* como *Daphne*[5] y su integración con bases de datos como *SQLite* o *PostgreSQL*, lo que permite adaptar la aplicación a distintos escenarios de uso.

2.2. Patrón *MVT*

El *framework Django* se fundamenta en el patrón de arquitectura *MVT (Model–View–Template)*[6], una adaptación del modelo clásico *MVC (Model–View–Controller)*[7] orientada al desarrollo web. Este patrón separa de forma clara la lógica de negocio, la gestión de los datos y la presentación visual, lo que permite un desarrollo estructurado, mantenible y escalable.

El patrón *MVT* está compuesto por tres elementos principales que colaboran de manera coordinada:

- El *Modelo (Model)* define la estructura de los datos y sus relaciones mediante clases de *Python*, que se traducen automáticamente en tablas dentro de la base de datos a través del sistema *ORM (Object–Relational Mapper)* de *Django*. Este enfoque permite manipular la información de forma sencilla sin necesidad de escribir consultas *SQL*, reduciendo errores y aumentando la productividad.

- La *Vista (View)* actúa como intermediaria entre el modelo y la plantilla. Su función es procesar las solicitudes del usuario, aplicar la lógica de negocio necesaria, recuperar los datos requeridos y devolver una respuesta adecuada. De esta forma, las vistas controlan el flujo de información dentro de la aplicación y determinan qué contenido debe mostrarse en cada momento.
- Por su parte, la *Plantilla (Template)* constituye la capa de presentación del sistema. Está formada por archivos *HTML* que utilizan el lenguaje de plantillas de *Django*, el cual permite insertar variables y estructuras de control para generar contenido dinámico. Gracias a este mecanismo, el aspecto visual de la aplicación se mantiene separado de la lógica, facilitando la modificación del diseño sin afectar al funcionamiento interno.

El flujo de trabajo del patrón *MVT* sigue un proceso sencillo y bien definido: el usuario realiza una petición *HTTPS*, *Django* identifica la vista correspondiente mediante la configuración de rutas del sistema, la vista obtiene y procesa los datos necesarios a través de los modelos, y finalmente se renderiza una plantilla que genera la respuesta *HTML* mostrada en el navegador.

En el proyecto *Verdad o Reto*, este patrón se aplica de forma completa. Los modelos representan las entidades principales del sistema (`Pack`, `Accion`, `Usuario` y `Sala Online`), las vistas gestionan la lógica del juego y la interacción con el usuario, y las plantillas definen la interfaz visible desde el navegador. Gracias a esta estructura, el código de la aplicación se mantiene limpio, modular y fácilmente ampliable. Además, el uso del patrón *MVT* ha permitido desarrollar el proyecto de manera iterativa, incorporando mejoras progresivas —como el sistema de autenticación, la gestión de *packs* o la personalización visual— sin comprometer la estabilidad general de la aplicación.

2.3. Tecnologías web

El desarrollo del *frontend* de la aplicación *Verdad o Reto* se ha llevado a cabo utilizando las tecnologías estándar de la web: *HTML*, *CSS* y *JavaScript*. Estas herramientas permiten definir la estructura, el estilo y la interactividad de la interfaz de usuario. El objetivo principal en el diseño del *frontend* ha sido crear una experiencia visual atractiva, intuitiva y coherente con el carácter lúdico del juego, garantizando al mismo tiempo una correcta visualización y usabilidad en todo tipo de dispositivos.

2.3.1. *HTML*

La estructura base de cada página se ha desarrollado con *HTML5*[8], siguiendo un enfoque semántico que mejora la accesibilidad y facilita la comprensión del código. Las plantillas de *Django*, ubicadas en el directorio de plantillas del proyecto, emplean etiquetas como `{% block content%}` y `{% endblock%}` para definir secciones dinámicas dentro de una plantilla principal, `base.html`, que actúa como marco común para el resto de las páginas.

Este enfoque modular permite mantener una estructura ordenada y coherente: los elementos compartidos (cabecera, pie de página, estilos globales o *scripts*) se definen una única vez en la plantilla base y se heredan en todas las demás páginas. De esta manera, se facilita el mantenimiento y la homogeneidad visual en toda la aplicación.

2.3.2. *CSS*

El lenguaje *CSS3 (Cascading Style Sheets)*[9] ha sido utilizado para definir la apariencia visual de la aplicación. *CSS* permite separar la presentación del contenido estructural, lo que favorece la reutilización del código y el mantenimiento a largo plazo. Además, proporciona herramientas avanzadas de diseño, como sistemas de rejilla (*CSS Grid*), cajas flexibles (*Flexbox*) y variables personalizadas, que facilitan la creación de interfaces consistentes y adaptables.

El uso de *CSS* ha permitido definir un estilo visual coherente con la identidad del proyecto, aplicar animaciones ligeras y garantizar una visualización uniforme en distintos navegadores. La modularización de las hojas de estilo —organizadas por componentes y secciones— contribuye a la claridad del código y a su fácil actualización.

2.3.3. Diseño *responsive*

El diseño *responsive*[10][11] (o diseño web adaptable) constituye un enfoque ampliamente adoptado en el desarrollo web actual, orientado a garantizar que una misma interfaz pueda visualizarse correctamente en dispositivos con diferentes tamaños de pantalla y resoluciones. Este paradigma evita la necesidad de mantener versiones separadas del sitio para cada tipo de dispositivo, favoreciendo una solución unificada y flexible.

Para lograr esta adaptabilidad, se emplean técnicas basadas en el uso de medidas relativas y reglas de estilo que permiten ajustar dinámicamente la disposición de los elementos según las características del dispositivo. De este modo, la interfaz reorganiza sus contenidos, tamaños y proporciones en función del espacio disponible.

En el caso de *Verdad o Reto*, la adopción de un diseño *responsive* garantiza que la aplicación sea plenamente funcional tanto en dispositivos móviles como en ordenadores de escritorio, manteniendo la legibilidad y la usabilidad del contenido. Esta decisión responde a uno de los objetivos fundamentales del proyecto: ofrecer una experiencia fluida y accesible en cualquier entorno de uso.

2.3.4. Interactividad y usabilidad

El lenguaje *JavaScript*[12] complementa las funcionalidades del *frontend*, permitiendo dotar de dinamismo e interactividad a la aplicación sin recargar el procesamiento del servidor. En este proyecto, se utiliza de manera controlada para mejorar la experiencia del usuario mediante

pequeñas animaciones, validaciones y manipulación puntual del contenido del *Document Object Model (DOM)*[13].

Asimismo, se han tenido en cuenta principios básicos de usabilidad y accesibilidad, orientados a garantizar una navegación intuitiva: contraste adecuado entre texto y fondo, elementos interactivos claramente identificables y una disposición visual que prioriza la legibilidad y la simplicidad. Estos aspectos contribuyen a una experiencia de usuario más agradable y accesible, en línea con los estándares actuales del diseño web.

2.4. Autenticación, autorización y seguridad

2.4.1. Gestión de usuarios

En aplicaciones web que manejan información personalizada o privada resulta fundamental disponer de mecanismos que permitan identificar a los usuarios y controlar su acceso a las distintas funcionalidades del sistema. En este contexto, la gestión de usuarios se apoya en dos conceptos clave y complementarios: la autenticación y la autorización.

La autenticación permite verificar la identidad de un usuario, normalmente mediante credenciales como un nombre de usuario y una contraseña, mientras que la autorización determina qué acciones puede realizar dicho usuario una vez ha sido autenticado. Estos mecanismos son especialmente relevantes en aplicaciones colaborativas, donde distintos usuarios pueden tener permisos diferentes sobre los recursos disponibles.

Django proporciona un sistema de autenticación integrado que incluye la gestión de usuarios, el almacenamiento seguro de contraseñas, el manejo de sesiones y un sistema de permisos extensible. Este sistema permite diferenciar entre usuarios autenticados y no autenticados, así como definir distintos niveles de acceso a las funcionalidades de la aplicación.

Asimismo, el *framework* contempla mecanismos para la gestión de contraseñas, incluyendo su modificación y recuperación en caso de olvido. Estas funcionalidades permiten al usuario restablecer sus credenciales de forma segura mediante procedimientos controlados, reforzando la protección de la identidad digital y contribuyendo a la fiabilidad global del sistema.

Gracias a este enfoque, es posible implementar escenarios habituales como la restricción de determinadas acciones a usuarios registrados, la edición colaborativa de contenidos o la separación entre áreas privadas y públicas dentro de la aplicación, garantizando un control coherente y seguro de los usuarios.

2.4.2. Protección *CSRF* y control de acceso

Las aplicaciones web están expuestas a distintos tipos de ataques que pueden comprometer la seguridad del sistema y la información de los usuarios. Uno de los ataques más comunes es el conocido como *Cross-Site Request Forgery (CSRF)*[14], que consiste en forzar a un usuario

autenticado a ejecutar acciones no deseadas sin su conocimiento.

Para mitigar este tipo de ataques, *Django* incorpora de forma nativa un sistema de protección *CSRF* basado en el uso de *tokens* únicos asociados a la sesión del usuario. Estos *tokens* se incluyen en los formularios y se verifican en el servidor antes de procesar cualquier petición que modifique el estado del sistema, garantizando que la solicitud proviene de una fuente legítima.

Además de los mecanismos frente a *CSRF*, resulta esencial implementar un control de acceso adecuado a las distintas vistas y funcionalidades de la aplicación. Este control permite restringir el acceso a determinadas acciones únicamente a usuarios autenticados o con los permisos necesarios, evitando accesos no autorizados a recursos sensibles.

La combinación de estos mecanismos contribuye a reforzar la seguridad global de la aplicación. De este modo, se establece una base sólida para el desarrollo de funcionalidades que impliquen la modificación de datos, la colaboración entre usuarios o la gestión de información privada.

2.5. Persistencia de datos

En aplicaciones web que gestionan información persistente resulta fundamental disponer de un sistema de almacenamiento de datos fiable y eficiente. En este proyecto, la persistencia de datos se aborda mediante el uso de sistemas de gestión de bases de datos relacionales, ampliamente utilizados en entornos web por su estabilidad y facilidad de integración.

2.5.1. *SQLite*

SQLite[15] es un sistema de gestión de bases de datos relacionales, ligero y de *software* libre que se caracteriza por no requerir un servidor independiente, ya que almacena toda la información en un único archivo. Esta característica lo convierte en una opción especialmente adecuada para entornos de desarrollo, pruebas y proyectos de pequeña o mediana escala.

En el contexto de aplicaciones desarrolladas con *Django*, *SQLite* se integra de forma nativa a través del *Object-Relational Mapper (ORM)* del *framework*, permitiendo el manejo de los datos de manera transparente sin necesidad de configuraciones adicionales complejas, lo que se traduce en no tener que escribir *SQL* directamente. Esto facilita un desarrollo rápido y reduce la carga administrativa durante las primeras fases del proyecto.

No obstante, *SQLite* presenta ciertas limitaciones en escenarios que requieren un alto nivel de concurrencia o un uso intensivo de operaciones de escritura, lo que puede afectar al rendimiento en aplicaciones con múltiples usuarios simultáneos. Por este motivo, su uso suele limitarse a entornos de desarrollo o pruebas, siendo sustituido por otros sistemas de bases de datos más robustos en producción.

2.5.2. *PostgreSQL*

PostgreSQL[16] es otro sistema de gestión de bases de datos relacionales de *software* libre, pero orientado a entornos de producción. Es reconocido por su robustez, escalabilidad y alto nivel de cumplimiento de los estándares *SQL*. Está diseñado para manejar grandes volúmenes de datos y un elevado número de conexiones concurrentes, lo que lo convierte en una opción adecuada para aplicaciones web con múltiples usuarios.

Django también ofrece soporte completo para *PostgreSQL* a través de su sistema *ORM*, permitiendo aprovechar sus funcionalidades avanzadas sin necesidad de escribir consultas *SQL* de forma directa. Esta integración facilita la portabilidad del proyecto entre distintos sistemas de bases de datos y mejora la mantenibilidad del código.

En el contexto de este proyecto, *PostgreSQL* se plantea como una base de datos muy adecuada para el entorno de producción, ya que proporciona mayor rendimiento, fiabilidad y capacidad de crecimiento frente a soluciones más ligeras. Su uso permite garantizar la integridad de los datos y soportar un uso concurrente más exigente, adaptándose a las necesidades de una aplicación web social con múltiples usuarios activos.

2.6. Acceso público mediante enlaces y códigos QR

2.6.1. Generación y uso de enlaces públicos

En el ámbito de las aplicaciones web sociales y colaborativas, el uso de enlaces públicos constituye un mecanismo habitual para facilitar el acceso rápido a determinados recursos o funcionalidades sin requerir un proceso previo de autenticación. Este mecanismo resulta especialmente útil en contextos donde se busca reducir la fricción de entrada y favorecer la participación de múltiples usuarios.

Los enlaces públicos permiten compartir contenidos o iniciar interacciones de forma sencilla mediante una URL única, que puede ser distribuida a través de distintos canales digitales. Este tipo de acceso es frecuente en aplicaciones orientadas al ocio, eventos o colaboración, donde la prioridad es la inmediatez y la facilidad de uso.

Desde el punto de vista de la seguridad, el uso de enlaces públicos suele apoyarse en identificadores únicos y difíciles de predecir, lo que limita el acceso únicamente a aquellos usuarios que disponen del enlace. De este modo, se logra un equilibrio entre accesibilidad y control, permitiendo ofrecer funcionalidades públicas sin comprometer la integridad del sistema ni la privacidad de los datos gestionados.

2.6.2. Generación de códigos QR

Los códigos QR (*Quick Response*)[17] se han consolidado como una solución ampliamente utilizada para facilitar el acceso rápido a recursos digitales desde dispositivos móviles. Estos

códigos permiten codificar información, como una URL, de forma que puede ser interpretada fácilmente mediante la cámara de un teléfono móvil o una aplicación lectora.

En aplicaciones web, los códigos QR suelen emplearse como una alternativa visual a los enlaces tradicionales, mejorando la experiencia de usuario en contextos presenciales o sociales. Su uso resulta especialmente adecuado cuando se desea compartir un recurso de manera inmediata, evitando la introducción manual de direcciones web y reduciendo posibles errores.

La generación de códigos QR asociados a enlaces públicos permite combinar ambos mecanismos, ofreciendo un acceso sencillo y directo a funcionalidades específicas de la aplicación. Esta estrategia es habitual en aplicaciones orientadas al ocio, eventos o juegos sociales. En este tipo de contexto, la facilidad de acceso son factores clave para fomentar la participación de los usuarios.

2.7. Comunicación en tiempo real y salas *online*

2.7.1. Comunicación en tiempo real en aplicaciones web

La comunicación en tiempo real se ha convertido en un elemento clave en el desarrollo de aplicaciones web modernas, especialmente en aquellas orientadas a la interacción social, la colaboración o el entretenimiento. Este tipo de comunicación permite el intercambio inmediato de información entre múltiples usuarios, posibilitando experiencias más dinámicas y participativas.

En el contexto de las aplicaciones web, la comunicación en tiempo real se apoya en tecnologías que permiten establecer conexiones persistentes entre el cliente y el servidor, reduciendo la latencia y evitando la necesidad de realizar peticiones continuas. Estas tecnologías hacen posible funcionalidades como videoconferencias, chats en vivo, salas virtuales o juegos multijugador, ampliamente utilizadas en plataformas sociales actuales.

La incorporación de mecanismos de comunicación en tiempo real resulta especialmente relevante en aplicaciones de tipo social o lúdico, ya que favorece la interacción simultánea entre los participantes y permite trasladar experiencias tradicionalmente presenciales al entorno digital. En este sentido, las salas *online* se presentan como una solución habitual para facilitar la participación remota de varios usuarios dentro de un mismo entorno virtual compartido.

2.7.2. *Jitsi Meet*

Jitsi Meet [18] es una plataforma de *software* libre orientada a la realización de videoconferencias y comunicación en tiempo real a través de la web. Está basada en tecnologías estándar como *WebRTC*, lo que permite su funcionamiento directamente desde el navegador sin necesidad de instalar *software* adicional por parte del usuario.

Entre las principales características de *Jitsi Meet* se encuentran la creación de salas virtuales, la comunicación mediante audio y vídeo, el intercambio de mensajes y la posibilidad de integrarse

en aplicaciones web de terceros. Al tratarse de un proyecto de *software* libre, ofrece además un alto grado de flexibilidad y personalización, lo que lo convierte en una opción atractiva para proyectos que requieren soluciones adaptables a necesidades concretas.

Jitsi Meet se utiliza ampliamente como alternativa a otras plataformas de videoconferencia comerciales, destacando por su enfoque en la privacidad y su facilidad de integración en entornos web. Estas características hacen que resulte especialmente adecuado para aplicaciones sociales y colaborativas. En este tipo de sistemas, *Jitsi Meet* permite incorporar salas *online* para la interacción simultánea de varios usuarios.

2.8. Opciones de despliegue y comparativa de plataformas

El despliegue de aplicaciones web es una fase fundamental en el ciclo de desarrollo, ya que permite poner el sistema a disposición de los usuarios en un entorno accesible, estable y seguro. En la actualidad, existen múltiples opciones para desplegar aplicaciones web, que abarcan desde servidores tradicionales autogestionados hasta plataformas en la nube que facilitan este proceso.

Dentro del paradigma de computación en la nube, las plataformas *Platform as a Service (PaaS)*[19] se han consolidado como una alternativa habitual para el despliegue de aplicaciones web, al ofrecer un entorno gestionado que abstrae al desarrollador de tareas relacionadas con la configuración del servidor, la gestión de dependencias o el escalado de la aplicación. Este tipo de plataformas resulta especialmente adecuado para proyectos académicos y aplicaciones de tamaño medio, donde la facilidad de uso y la rapidez de despliegue son factores clave.

A la hora de seleccionar un entorno de publicación, suelen considerarse diversos criterios, como la compatibilidad con el *framework* utilizado, el soporte para bases de datos relacionales, la gestión de archivos estáticos, la posibilidad de configurar dominios y certificados *HTTPS*, así como el coste asociado al servicio. Además, aspectos como la escalabilidad, la fiabilidad y la disponibilidad del sistema juegan un papel importante en la elección final.

Entre las plataformas de despliegue más utilizadas para aplicaciones web desarrolladas con *Django* se encuentran diversas soluciones en la nube que permiten una integración sencilla con servicios externos y facilitan el mantenimiento del sistema. La elección de una plataforma concreta dependerá de las necesidades específicas del proyecto, así como de los requisitos técnicos y funcionales definidos durante la fase de análisis.

En el presente proyecto se ha optado por la plataforma *Render*[20], que ofrece un entorno *PaaS* compatible con aplicaciones desarrolladas en *Django*. Esta plataforma permite desplegar la aplicación directamente desde un repositorio de código, gestionar variables de entorno de forma segura, integrar bases de datos relacionales como *PostgreSQL* y configurar certificados *HTTPS* de manera automática. La elección de *Render* responde a su equilibrio entre simplicidad de configuración, compatibilidad tecnológica y adecuación al alcance académico del proyecto, proporcionando un entorno estable y fácilmente mantenible.

Capítulo 3

Análisis y diseño

El presente capítulo aborda el análisis y diseño del sistema desarrollado. A partir de las tecnologías y fundamentos descritos previamente, se definen los actores implicados, los casos de uso principales y la estructura del modelo de datos, así como la arquitectura general adoptada.

Este análisis permite establecer una base sólida para la implementación posterior, asegurando la coherencia entre los requisitos funcionales del sistema y las decisiones técnicas adoptadas.

3.1. Casos de uso

3.1.1. Actores

En el sistema desarrollado se identifican distintos tipos de actores en función de su nivel de acceso y de las funcionalidades disponibles para cada uno de ellos. Estos actores representan los diferentes perfiles de usuario que interactúan con la aplicación web y sirven como base para la definición de los casos de uso del sistema.

- **Usuario invitado:** Representa a aquellos usuarios que acceden a la aplicación sin necesidad de registrarse, normalmente a través de enlaces públicos o códigos QR. Este actor puede participar en el juego y acceder a vistas públicas del sistema, pero no dispone de permisos para modificar contenido ni gestionar recursos persistentes. En cualquier momento, el usuario invitado tiene la posibilidad de registrarse para acceder a funcionalidades adicionales.
- **Usuario registrado:** Corresponde a aquellos usuarios que disponen de una cuenta en el sistema y han iniciado sesión correctamente. Este actor puede participar en el juego y gestionar sus propios *packs* personalizados de preguntas y retos, así como acceder a funcionalidades avanzadas como la creación de salas *online*. Adicionalmente, un usuario registrado puede actuar como colaborador en *packs* de otros usuarios, disponiendo en ese caso de permisos de edición limitados sobre dichos recursos. Asimismo, puede gestionar sus credenciales de acceso, incluyendo la modificación o recuperación de la contraseña en caso de olvido.

- **Administrador**: Dispone de todos los privilegios asociados al usuario registrado y, además, puede realizar tareas de supervisión, gestión y mantenimiento global del sistema mediante el panel de administración proporcionado por Django.

La diferenciación entre estos actores permite definir de forma precisa los casos de uso del sistema y establecer mecanismos de control de acceso adecuados, garantizando una interacción segura, coherente y acorde a los permisos de cada perfil de usuario.

3.1.2. Supuestos y límites

Durante el análisis y diseño del sistema se han considerado una serie de supuestos y límites que permiten acotar el alcance del proyecto y definir el contexto en el que opera la aplicación web.

Se asume que los usuarios acceden al sistema a través de navegadores web modernos y disponen de una conexión a Internet estable, requisito imprescindible para el correcto funcionamiento de la aplicación y, en especial, para el uso de funcionalidades de comunicación en tiempo real. Asimismo, se presupone que los usuarios cuentan con conocimientos básicos para interactuar con aplicaciones web y utilizar dispositivos móviles o de escritorio.

El sistema está diseñado para ofrecer una experiencia de juego social y colaborativa; sin embargo, no incluye mecanismos automáticos de moderación del contenido generado por los usuarios ni de supervisión del comportamiento durante las sesiones de juego *online*. Del mismo modo, la aplicación no contempla el funcionamiento en modo *offline* ni la sincronización de datos en ausencia de conexión a Intenet.

Por último, el proyecto depende de servicios externos para la implementación de determinadas funcionalidades, como la comunicación en tiempo real o el envío de correos electrónicos transaccionales. Esto implica que el correcto funcionamiento de estas características está sujeto a la disponibilidad, rendimiento y limitaciones propias de las plataformas utilizadas.

Estas decisiones permiten centrar el desarrollo en los objetivos principales del proyecto y mantener un alcance adecuado y realista para un trabajo de fin de grado.

3.1.3. Diagrama de casos de uso

Con el objetivo de representar de forma visual las interacciones entre los actores identificados y las funcionalidades principales del sistema, se ha elaborado un diagrama de casos de uso. Este tipo de diagramas permite describir, a alto nivel, las acciones que puede realizar cada actor dentro de la aplicación, facilitando la comprensión del comportamiento global del sistema y sirviendo como base para las decisiones de diseño posteriores.

En la Figura 3.1 se muestra el diagrama de casos de uso correspondiente a la aplicación, en el que se recogen los casos de uso principales relacionados con la gestión de usuarios, la creación y edición de *packs*, el acceso público mediante enlaces o códigos QR, así como la creación de salas *online* para la interacción remota.

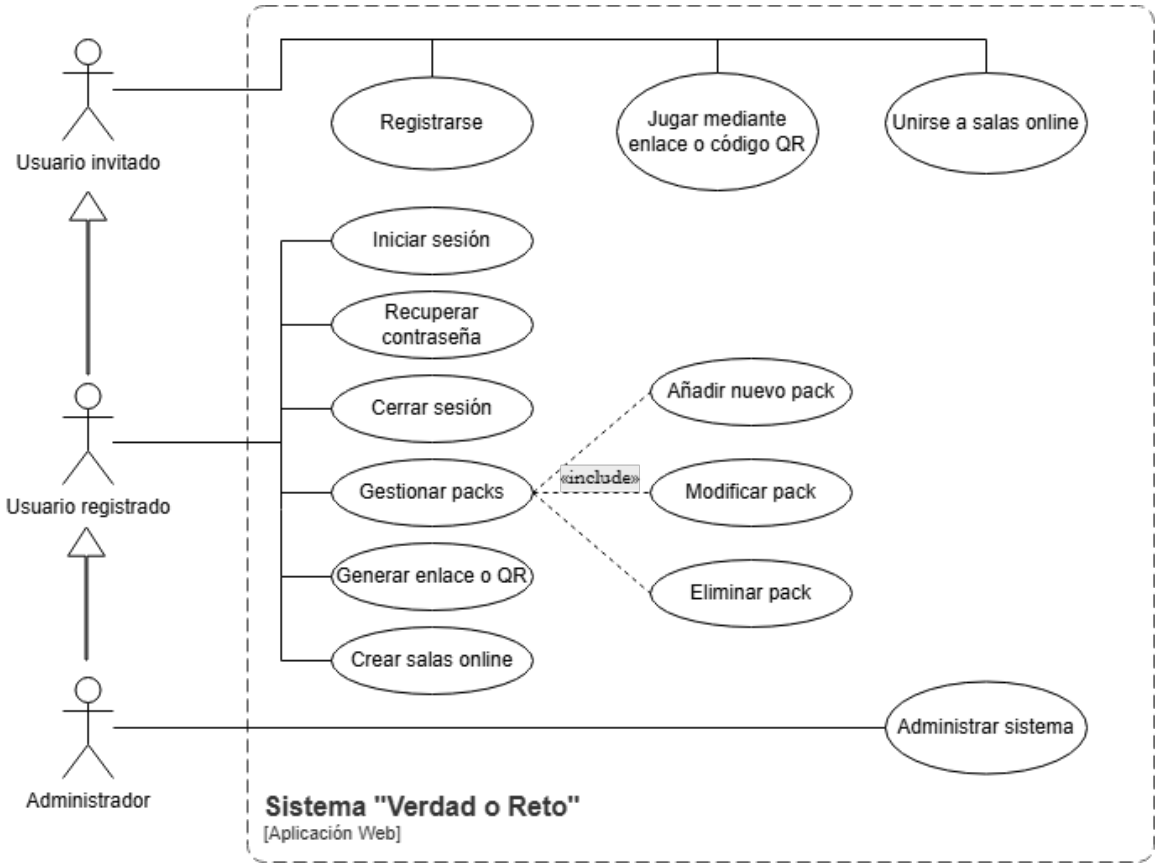

Figura 3.1: Diagrama de casos de uso del sistema.

Fuente: elaboración propia.

El diagrama refleja la diferenciación entre usuarios invitados y usuarios registrados, así como la existencia de un perfil de administrador con capacidades de supervisión y mantenimiento del sistema mediante el panel de administración. La descripción detallada de cada caso de uso, incluyendo sus objetivos y condiciones principales, se desarrolla en el apartado siguiente.

3.1.4. Descripción de los casos de uso

A continuación, se describen los principales casos de uso del sistema, identificados a partir del análisis de los actores y representados en el diagrama de casos de uso. Estas descripciones permiten detallar las funcionalidades esenciales de la aplicación y el modo en que los distintos actores interactúan con el sistema.

Registrarse

El proceso de registro es iniciado por un usuario invitado que desea crear una cuenta en el sistema. Para ello, el usuario proporciona la información requerida, que es validada por el sistema antes de proceder a la creación de una nueva cuenta. Una vez completado el registro con éxito, el usuario obtiene acceso a las funcionalidades reservadas a los usuarios registrados.

Iniciar sesión

El inicio de sesión permite a los usuarios registrados acceder al sistema mediante la introducción de sus credenciales de autenticación. El sistema verifica la información proporcionada y, en caso de ser válida, establece una sesión autenticada que habilita el acceso a las funcionalidades disponibles según los permisos asociados al usuario.

Recuperar contraseña

Este caso de uso permite a un usuario registrado restablecer su contraseña en caso de que la haya olvidado. El proceso se inicia mediante la solicitud de recuperación desde la interfaz de autenticación. El sistema valida la solicitud y habilita un procedimiento seguro que permite al usuario definir una nueva contraseña. Una vez completado el proceso, el usuario puede acceder nuevamente al sistema utilizando sus nuevas credenciales.

Cerrar sesión

Mediante este caso de uso, un usuario registrado puede finalizar su sesión activa en cualquier momento. El sistema invalida la sesión autenticada, asegurando la protección del acceso y devolviendo al usuario a un estado no autenticado.

Jugar mediante enlace o código QR

Este caso de uso permite a los usuarios invitados acceder a un *pack* de juego a través de un enlace público o un código QR. El sistema muestra el contenido asociado al *pack* y posibilita la participación en la partida sin necesidad de registro ni autenticación previa.

Gestionar *packs*

La gestión de *packs* permite a un usuario registrado crear nuevos conjuntos de preguntas y retos, así como modificar o eliminar aquellos ya existentes. El sistema registra los cambios realizados y aplica los mecanismos de control de acceso necesarios, de modo que únicamente el propietario del *pack* o los colaboradores autorizados pueden editar su contenido.

Generar enlace o código QR

A través de este caso de uso, un usuario registrado puede generar un enlace público o un código QR asociado a uno de sus *packs*. El sistema crea un identificador único que puede ser compartido con otros usuarios, facilitando el acceso directo al contenido del *pack*.

Crear salas *online*

La creación de salas *online* permite a un usuario registrado iniciar una sesión de juego asociada a un *pack* determinado. El sistema genera la sala y proporciona un enlace de acceso que posibilita la participación simultánea de varios usuarios, tanto registrados como invitados, durante la partida.

Unirse a salas *online*

Este caso de uso contempla el acceso a una sala *online* existente mediante un enlace, pudiendo ser iniciado tanto por usuarios registrados como por usuarios invitados. El sistema permite la incorporación a la sesión y habilita la interacción remota entre los participantes.

Administrar sistema

La administración del sistema corresponde al actor administrador, que dispone de funcionalidades orientadas a la supervisión, gestión y mantenimiento global de la aplicación. A través del panel de administración, puede gestionar usuarios, *packs* y otros elementos del sistema necesarios para garantizar su correcto funcionamiento.

3.2. Modelo de datos

El modelo de datos define la estructura de la información gestionada por el sistema, especificando las entidades principales, sus atributos y las relaciones existentes entre ellas. Un diseño adecuado del modelo de datos resulta fundamental para garantizar la coherencia, integridad y mantenibilidad de la aplicación, sirviendo como base para la posterior implementación y persistencia de la información.

3.2.1. Entidades principales

El sistema desarrollado gestiona distintos tipos de información relacionados con los usuarios, el contenido del juego y las interacciones entre los participantes. Para ello, se ha definido un conjunto de entidades principales que permite estructurar y organizar los datos de forma coherente, normalizada y flexible.

- **Usuario**: representa a las personas que interactúan con la aplicación y constituye la entidad central del sistema. Almacena la información necesaria para la identificación y autenticación de los usuarios registrados, así como los datos asociados a su rol dentro del sistema, ya sea como usuario estándar o administrador.
- ***Pack***: corresponde a un conjunto de preguntas y retos creados por un usuario registrado. Cada *pack* agrupa contenido relacionado y puede ser gestionado por su propietario o por otros usuarios que dispongan de permisos de colaboración.

- **Acción (Pregunta/Reto)**: representa los elementos individuales que conforman un *pack*. Cada acción pertenece a un único *pack* y define el contenido que se presenta a los usuarios durante el desarrollo del juego, diferenciándose mediante un atributo que indica si se trata de una pregunta o de un reto.
- **Colaboración**: actúa como entidad asociativa entre usuarios y *packs*, modelando la participación de usuarios colaboradores en la edición de contenido que no es de su propiedad. Esta entidad permite gestionar de forma explícita los permisos de acceso y edición asignados a cada colaborador, así como registrar información adicional asociada a la colaboración.
- **Sala *online***: representa una sesión de juego activa que permite la interacción simultánea de varios participantes. Cada sala se asocia a un *pack* concreto y facilita la gestión del acceso y de la comunicación en tiempo real entre los usuarios durante el transcurso de la partida.

La identificación de estas entidades constituye la base para la definición del modelo de datos del sistema, que se detalla en los apartados siguientes mediante el análisis de sus relaciones y atributos.

3.2.2. Diagrama entidad-relación

Con el fin de representar de manera gráfica la estructura del modelo de datos del sistema, se ha elaborado un diagrama entidad–relación. Este tipo de diagramas permite visualizar las entidades principales identificadas durante el análisis, así como las relaciones existentes entre ellas, facilitando la comprensión de la organización de la información gestionada por la aplicación.

En la Figura 3.2 se presenta el diagrama entidad–relación correspondiente al sistema desarrollado. En él se reflejan las entidades relacionadas con los usuarios, los *packs* de juego, las acciones que los componen (preguntas y retos) y las salas *online*, junto con las distintas relaciones que se establecen entre dichas entidades.

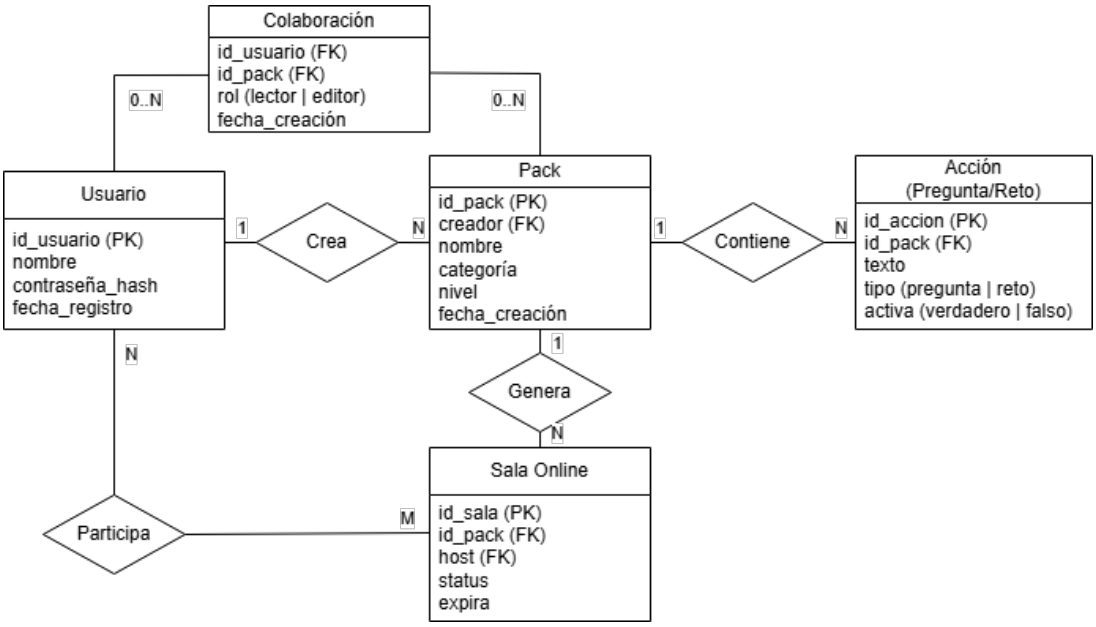

Figura 3.2: Diagrama entidad–relación del sistema.

Fuente: elaboración propia.

El diagrama pone de manifiesto la relación entre usuarios y *packs*, distinguiendo claramente entre el usuario creador de un *pack* y aquellos usuarios que participan como colaboradores en su edición. Esta relación de colaboración se modela mediante una entidad intermedia, lo que permite representar de forma explícita los permisos asociados a cada usuario colaborador.

Asimismo, se representa la asociación entre los *packs* y las acciones que los conforman, reflejando la estructura jerárquica del contenido del juego. Del mismo modo, se modela la relación entre los *packs* y las salas *online*, permitiendo representar las sesiones de juego activas asociadas a un contenido concreto.

El detalle de los atributos y campos que componen cada una de las entidades definidas en el modelo se describe en el siguiente apartado.

3.2.3. Campos y relaciones

Una vez definidas las entidades principales del sistema y representadas sus relaciones mediante el diagrama entidad–relación, se describen a continuación los campos más relevantes asociados a cada entidad, así como la naturaleza de las relaciones existentes entre ellas. Esta descripción permite comprender con mayor detalle la estructura de la información gestionada por la aplicación.

Usuario

En la entidad *Usuario* se almacena la información necesaria para la identificación y autenticación de las personas que interactúan con el sistema. Entre sus campos más relevantes se incluyen

un identificador único, el nombre del usuario, las credenciales necesarias de acceso y la fecha de registro.

Desde el punto de vista relacional, un usuario puede crear múltiples *packs*, colaborar en *packs* creados por otros usuarios mediante la entidad *Colaboración* y participar en varias salas *online*.

Pack

Cada *pack* dispone de un identificador propio, un nombre, una categoría, un nivel y una fecha de creación, y se encuentra asociado a un único usuario creador.

Un *pack* puede contener múltiples acciones y generar varias salas *online* a lo largo de su ciclo de vida. Además, puede ser editado por usuarios colaboradores autorizados, de acuerdo con los permisos definidos en la entidad *Colaboración*.

Acción (Pregunta/Reto)

La entidad *Acción* incluye campos como el texto asociado, el tipo de acción (pregunta o reto) y un indicador que determina si se encuentra activa. Cada acción pertenece a un único *pack*, lo que permite estructurar el contenido del juego de forma jerárquica.

Esta organización facilita la reutilización del contenido y el mantenimiento de los *packs*.

Colaboración

La entidad *Colaboración* contiene las referencias al usuario colaborador y al *pack* asociado, junto con el rol o permiso asignado y la fecha de creación de la colaboración.

Esta estructura permite gestionar de forma explícita los permisos de acceso y edición sobre el contenido compartido, diferenciando claramente entre el creador de un *pack* y los usuarios que colaboran en su edición.

Sala *online*

Cada sala *online* cuenta con un identificador, un estado y una fecha de expiración. Se encuentra vinculada tanto al *pack* utilizado como al usuario que actúa como anfitrión.

Las salas tienen un carácter temporal y permiten la participación simultánea de múltiples usuarios durante el desarrollo de una partida.

Relaciones entre entidades

Las relaciones definidas en el modelo de datos establecen que un usuario puede crear varios *packs*, mientras que cada *pack* tiene un único creador. La colaboración entre usuarios y *packs* se modela mediante una entidad asociativa, lo que permite representar de forma explícita los permisos de edición compartida del contenido.

Asimismo, existe una relación de tipo uno a muchos entre los *packs* y las acciones que los componen, así como entre los *packs* y las salas *online* que pueden generarse. Por último, la relación de participación permite que múltiples usuarios se unan a una misma sala *online*. Estas relaciones garantizan la coherencia del modelo de datos del sistema.

3.3. Arquitectura del sistema

3.3.1. Visión general

La aplicación desarrollada sigue una arquitectura web de tipo cliente–servidor, en la que la interacción de los usuarios se realiza a través de un navegador web que actúa como cliente y se comunica con un servidor central encargado de procesar las peticiones y gestionar la lógica del sistema. Este enfoque permite ofrecer una aplicación accesible desde distintos dispositivos y plataformas, sin necesidad de instalar *software* adicional.

El servidor está implementado utilizando el *framework Django*, que actúa como núcleo del sistema y concentra la gestión de la lógica de negocio, las vistas de la aplicación y los mecanismos de autenticación y autorización. La información persistente se almacena en una base de datos relacional, a la que se accede mediante el sistema *ORM* proporcionado por *Django*, lo que facilita la gestión de los datos y refuerza la independencia entre la lógica de la aplicación y la capa de persistencia.

Asimismo, la arquitectura contempla la integración de servicios externos para la implementación de funcionalidades específicas, como la comunicación en tiempo real mediante salas *online* y el envío de correos electrónicos transaccionales asociados a procesos de autenticación. Estos servicios se comunican con la aplicación web a través de interfaces bien definidas, permitiendo ampliar las capacidades del sistema de forma modular y sin comprometer su estructura interna.

Esta visión general sienta las bases para una arquitectura modular y escalable, facilitando el mantenimiento del sistema y la incorporación de nuevas funcionalidades en el futuro.

3.3.2. Diagrama de arquitectura

Con el objetivo de representar de forma visual la arquitectura general del sistema y la interacción entre sus distintos componentes, se ha elaborado el diagrama de arquitectura que se muestra en la Figura 3.3. Este diagrama permite identificar los elementos principales que conforman la aplicación, así como los flujos de comunicación establecidos entre ellos.

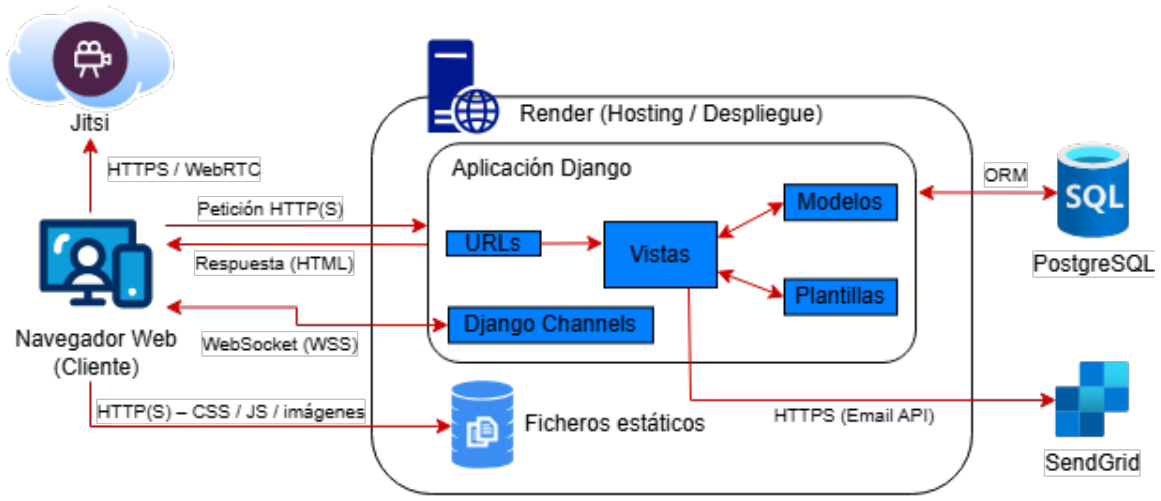

Figura 3.3: Diagrama de arquitectura general del sistema.

Fuente: elaboración propia.

En el lado del cliente se sitúa el navegador web, a través del cual los usuarios interactúan con la aplicación. El navegador realiza peticiones *HTTPS* al servidor de la aplicación y recibe como respuesta el contenido generado, incluyendo las vistas *HTML* y los recursos estáticos necesarios para el correcto funcionamiento de la interfaz, como hojas de estilo, *scripts* y elementos gráficos.

La aplicación web se encuentra desplegada en la plataforma *Render*, que actúa como entorno de alojamiento y ejecución del servicio. En su interior se ejecuta la aplicación desarrollada con el *framework Django*, que constituye el núcleo del sistema. *Django* se encarga de gestionar el enrutado de las peticiones mediante el sistema de URLs, la ejecución de la lógica de negocio a través de las vistas y la generación de las respuestas utilizando el sistema de plantillas.

La persistencia de la información se gestiona mediante una base de datos relacional *PostgreSQL*, accesible desde la aplicación a través del sistema *ORM* proporcionado por *Django*. Esta capa de abstracción permite desacoplar la lógica de la aplicación del sistema gestor de bases de datos, facilitando el mantenimiento y la evolución del modelo de datos.

Para las funcionalidades que requieren comunicación en tiempo real, como la gestión de sesiones de juego activas, el sistema incorpora *Django Channels*, que permite establecer conexiones *WebSocket* seguras (*WSS*) entre el navegador y el servidor. Este mecanismo complementa el modelo de comunicación basado en *HTTP(S)* y permite una interacción más dinámica entre los usuarios y la aplicación.

Asimismo, la arquitectura integra un servicio externo de videocomunicación basado en *Jitsi* para la implementación de las salas *online*. En este caso, la comunicación de audio y vídeo se establece directamente entre el navegador del usuario y el servicio *Jitsi* mediante tecnologías *WebRTC*, evitando que este tráfico pase por el servidor de la aplicación. De este modo, *Django* actúa como elemento de orquestación, proporcionando la información necesaria para el acceso a las salas, pero sin asumir la carga asociada a la transmisión multimedia.

De forma complementaria, se incorpora un servicio externo de envío de correo electrónico

para la gestión de notificaciones transaccionales asociadas al proceso de restablecimiento de contraseña. En este caso, la aplicación *Django* se comunica con dicho servicio mediante peticiones *HTTPS* a través de una interfaz de programación de aplicaciones (*API*), delegando en él la entrega efectiva de los mensajes a los usuarios.

En conjunto, el diagrama de arquitectura pone de manifiesto una separación clara de responsabilidades entre los distintos componentes del sistema, favoreciendo una arquitectura modular, escalable y alineada con las buenas prácticas en el desarrollo de aplicaciones web.

3.3.3. Capas del sistema

La arquitectura del sistema se organiza en distintas capas, cada una de ellas con responsabilidades bien definidas. Esta separación por capas permite estructurar la aplicación de forma clara, facilitar su mantenimiento y favorecer la escalabilidad y evolución del sistema a largo plazo.

Capa de presentación

La capa de presentación es la encargada de la interacción directa con los usuarios y se materializa a través del navegador web. En esta capa se muestran las vistas *HTML* generadas por el servidor, así como los recursos estáticos asociados, como hojas de estilo, *scripts* y elementos gráficos.

El navegador se comunica con el servidor mediante peticiones *HTTPS* para la carga de contenido y, en el caso de funcionalidades que requieren actualización en tiempo real, mediante conexiones *WebSocket* seguras (*WSS*). Asimismo, esta capa establece comunicación directa con servicios externos, como *Jitsi*, para la gestión de audio y vídeo durante las salas *online*.

Capa de aplicación

La capa de aplicación está implementada mediante el *framework Django* y constituye el núcleo funcional del sistema. En ella se gestiona el enrutado de las peticiones a través del sistema de URLs, la ejecución de la lógica de negocio mediante las vistas y la generación de las respuestas utilizando el sistema de plantillas.

Esta capa también es responsable de coordinar las funcionalidades en tiempo real a través de *Django Channels*, permitiendo la gestión de eventos y estados compartidos durante las sesiones de juego. Además, actúa como elemento de orquestación para la integración de servicios externos, proporcionando al cliente la información necesaria para su uso sin asumir directamente la carga de procesamiento asociada.

Capa de persistencia

La capa de persistencia se encarga del almacenamiento y gestión de la información del sistema. Está basada en una base de datos relacional, accesible desde la capa de aplicación mediante el sistema *ORM* de *Django*.

Esta capa almacena los datos relativos a usuarios, *packs*, acciones, colaboraciones y salas *online*, garantizando la coherencia e integridad de la información. El uso del ORM permite abstraer el acceso a la base de datos, reduciendo el acoplamiento entre la lógica de la aplicación y el sistema gestor de bases de datos.

Capa de servicios externos

El sistema integra una capa de servicios externos destinada a cubrir funcionalidades específicas que no se implementan directamente en la aplicación. En este proyecto, esta capa incluye el servicio de videocomunicación *Jitsi*, utilizado para la creación de salas *online* con comunicación de audio y vídeo en tiempo real, así como un servicio de envío de correo electrónico para la gestión de notificaciones transaccionales asociadas al proceso de recuperación de contraseña.

La comunicación con estos servicios se realiza de forma desacoplada y varía en función de la naturaleza de cada integración. En el caso de *Jitsi*, la conexión se establece directamente entre el cliente y el servicio externo mediante tecnologías *WebRTC*. Por el contrario, en el envío de correos electrónicos, la comunicación se produce entre la capa de aplicación y el servicio externo a través de peticiones *HTTPS* basadas en una *API*. Este enfoque permite mantener separadas las responsabilidades de cada componente, favoreciendo una arquitectura modular y escalable.

En conjunto, la organización del sistema en capas refuerza la separación de responsabilidades y permite una arquitectura modular, alineada con las buenas prácticas en el desarrollo de aplicaciones web.

3.3.4. Flujo de información

El flujo de información del sistema describe el recorrido que siguen los datos desde que el usuario interactúa con la aplicación hasta que el sistema procesa la solicitud y devuelve una respuesta, así como la comunicación establecida entre los distintos componentes de la arquitectura. Este flujo se apoya en la arquitectura cliente–servidor y en la separación por capas descrita en los apartados anteriores.

El proceso se inicia cuando el usuario realiza una acción desde el navegador web, como acceder a un *pack*, iniciar una partida o crear una sala *online*. Estas acciones generan peticiones *HTTP(S)* que son enviadas al servidor de la aplicación, donde son recibidas y gestionadas por el sistema de enrutado de *Django*.

Una vez procesada la petición, la capa de aplicación ejecuta la lógica correspondiente a través de las vistas, validando permisos, gestionando el estado de la sesión y accediendo a los datos necesarios mediante el *ORM* de *Django*. En caso de ser necesario, la aplicación consulta o modifica la información persistente almacenada en la base de datos relacional.

Tras el procesamiento de la petición, el servidor genera la respuesta adecuada, que puede consistir en una vista *HTML* renderizada, datos estructurados o una redirección a otra funcionalidad del sistema. Esta respuesta es enviada de vuelta al navegador, donde se presenta al usuario junto con los recursos estáticos necesarios para su correcta visualización.

Para aquellas funcionalidades que requieren interacción en tiempo real, como la actualización del estado de una sala de juego, el flujo de información se apoya en conexiones *WebSocket* gestionadas mediante *Django Channels*. Estas conexiones permiten intercambiar información de forma bidireccional entre el cliente y el servidor sin necesidad de realizar nuevas peticiones *HTTP*, proporcionando una experiencia más dinámica.

En el caso de las salas *online* con comunicación de audio y vídeo, el flujo de información se establece directamente entre el navegador del usuario y el servicio externo de videocomunicación *Jitsi* mediante tecnologías *WebRTC*. El servidor de la aplicación actúa como elemento de coordinación, proporcionando los datos necesarios para el acceso a la sala, pero sin intervenir en la transmisión del contenido multimedia.

Por otro lado, en el caso de los procesos de recuperación de contraseña, el flujo de información incluye la comunicación entre la capa de aplicación y un servicio externo de envío de correo electrónico. Tras recibir la solicitud del usuario, el servidor genera el mensaje correspondiente y lo transmite al servicio externo mediante una petición *HTTPS* basada en una *API*. Dicho servicio se encarga de la entrega efectiva del mensaje al destinatario, completando así el proceso sin que intervenga directamente el cliente en esta fase.

En conjunto, el flujo de información del sistema combina mecanismos síncronos basados en *HTTP(S)* con comunicaciones asíncronas y en tiempo real, garantizando una interacción fluida entre los usuarios y una correcta separación de responsabilidades entre los distintos componentes de la arquitectura.

3.3.5. Integración de sistemas internos

La integración de los distintos sistemas internos de la aplicación se basa en una comunicación estructurada y coherente entre los componentes que conforman la arquitectura, garantizando el correcto funcionamiento del sistema como un conjunto unificado.

El núcleo de esta integración se encuentra en la aplicación desarrollada con *Django*, que actúa como punto central de coordinación. A través del sistema de enrutado, las peticiones recibidas desde el cliente se dirigen a las vistas correspondientes, donde se ejecuta la lógica de aplicación y se gestionan las interacciones entre los distintos módulos del sistema, incluyendo los mecanismos de autenticación y autorización.

La comunicación entre la capa de aplicación y la capa de persistencia se realiza mediante el sistema *ORM* de *Django*, que permite acceder y manipular los datos almacenados en la base de datos relacional de forma transparente. Este enfoque facilita la integración entre la lógica de la aplicación y el modelo de datos, manteniendo la independencia entre ambas capas y asegurando la coherencia de la información.

Para las funcionalidades que requieren comunicación en tiempo real, la integración interna se extiende a través de *Django Channels*, que permite gestionar conexiones *WebSocket* y coordinar eventos compartidos entre usuarios. De este modo, los distintos componentes del sistema pueden intercambiar información de forma asíncrona sin interferir con el flujo principal de peticiones HTTP/HTTPS.

Asimismo, la integración entre los distintos módulos del sistema se apoya en una separación clara de responsabilidades, de manera que cada componente se encarga de una funcionalidad específica. Este enfoque modular facilita el mantenimiento del código, la detección de errores y la incorporación de nuevas funcionalidades sin afectar al resto del sistema.

En conjunto, la integración de los sistemas internos garantiza una comunicación eficiente entre los distintos componentes de la aplicación, reforzando la cohesión del sistema y contribuyendo a una arquitectura robusta y escalable.

3.4. Diseño de la interfaz y navegación

3.4.1. Estructura de navegación

La estructura de navegación del sistema se ha diseñado con el objetivo de ofrecer una experiencia de uso clara intuitiva y coherente, permitiendo a los usuarios acceder de forma sencilla a las distintas funcionalidades de la aplicación en función de su perfil y nivel de acceso. Para ello, se ha establecido una diferenciación explícita entre las vistas públicas y las vistas privadas del sistema, facilitando la comprensión del flujo de navegación desde el primer acceso.

Las vistas públicas permiten el acceso a funcionalidades básicas sin necesidad de autenticación previa, incluyendo el registro de nuevos usuarios y los procesos de recuperación de credenciales. A través de estas vistas, los usuarios invitados pueden acceder a *packs* de juego mediante enlaces públicos o códigos QR, participar en partidas y unirse a salas *online*. Este enfoque reduce las barreras de entrada al sistema y favorece una incorporación rápida de nuevos usuarios, priorizando el acceso inmediato a la experiencia de juego.

Por otro lado, las vistas privadas están reservadas a usuarios registrados que han iniciado sesión en el sistema. Desde estas secciones, el usuario puede gestionar sus propios *packs* de preguntas y retos, generar enlaces o códigos QR para compartir contenido, crear salas *online* y acceder al conjunto completo de funcionalidades avanzadas de la aplicación. El acceso a estas vistas se controla mediante mecanismos de autenticación y autorización, garantizando que cada usuario solo pueda acceder a los recursos acordes a su perfil.

La navegación entre las distintas secciones de la aplicación se realiza principalmente mediante menús y enlaces claramente identificados, manteniendo una estructura coherente en todas las páginas. Esta organización permite que el usuario comprenda en todo momento su ubicación dentro del sistema y facilite el acceso al resto de funcionalidades disponibles, favoreciendo una navegación fluida, predecible y alineada con los principios de usabilidad.

3.4.2. Plantillas principales

La interfaz de usuario del sistema se ha implementado mediante un conjunto de plantillas *HTML* organizadas de forma modular, con el objetivo de garantizar la reutilización de elementos comunes y mantener una estructura visual coherente en todas las vistas de la aplicación. Este enfoque permite separar la estructura general de la interfaz del contenido específico de cada página, facilitando el mantenimiento y la evolución del sistema.

Para ello, se ha definido una plantilla base que actúa como punto de partida para el resto de páginas. Esta plantilla incluye los elementos compartidos por toda la aplicación, como la estructura general de la página, los menús de navegación y la inclusión de los recursos comunes (hojas de estilo, *scripts* y otros elementos estáticos). A partir de esta base, el resto de plantillas se construyen mediante mecanismos de herencia, permitiendo personalizar el contenido de cada vista sin necesidad de duplicar código.

Sobre esta estructura común se han desarrollado plantillas específicas orientadas a las funcionalidades principales del sistema. Entre ellas se incluyen las vistas públicas de acceso a *packs* y salas *online*, las vistas relacionadas con la autenticación y gestión de credenciales (registro, inicio de sesión y recuperación de contraseña), así como las vistas privadas destinadas a la gestión de *packs*, la creación de salas y la administración del contenido por parte de los usuarios registrados.

El uso de plantillas reutilizables y una jerarquía bien definida permite mantener una apariencia visual consistente en toda la aplicación y simplifica las tareas de mantenimiento, ya que cualquier modificación en los elementos comunes puede aplicarse de forma centralizada sin afectar al resto de vistas.

3.4.3. Diseño visual y estilo

El diseño visual de la aplicación se ha concebido con el objetivo de ofrecer una interfaz atractiva, clara y alineada con la naturaleza social y lúdica del proyecto. Para ello, se ha optado por un estilo visual de inspiración neón, caracterizado por el uso de colores intensos y contrastados sobre fondos oscuros, que refuerza la identidad digital y dinámica del sistema.

Este enfoque visual permite destacar de forma eficaz los elementos interactivos más relevantes, como botones y acciones principales, facilitando su identificación por parte del usuario. Al mismo tiempo, se ha mantenido un diseño limpio y ordenado, aplicando principios de minimalismo en

la disposición de los elementos, con el fin de evitar la sobrecarga visual y priorizar la legibilidad de los contenidos.

La interfaz mantiene una línea gráfica homogénea en todas las vistas de la aplicación, apoyada en una paleta de colores coherente y una tipografía clara que favorece la lectura tanto en dispositivos de escritorio como en dispositivos móviles. Esta coherencia visual contribuye a mejorar la experiencia de usuario y a reforzar la identidad del sistema.

En conjunto, el uso del estilo neón aporta una estética atractiva y acorde al contexto del juego social *Verdad o Reto*, manteniendo un equilibrio adecuado entre impacto visual, usabilidad y claridad en la presentación de la información.

3.4.4. Adaptabilidad y *responsive design*

La aplicación ha sido diseñada teniendo en cuenta la diversidad de dispositivos desde los que puede ser utilizada, prestando especial atención al acceso desde dispositivos móviles. Para ello, se ha adoptado un enfoque de diseño adaptable (*responsive design*), que permite que la interfaz se ajuste de forma dinámica al tamaño y a la orientación de la pantalla del dispositivo.

La disposición de los elementos visuales se adapta automáticamente en función del ancho disponible, reorganizando los contenidos con el objetivo de mantener la legibilidad y la facilidad de uso en pantallas de tamaño reducido. De este modo, se garantiza que las funcionalidades principales del sistema sean accesibles tanto desde ordenadores de escritorio como desde teléfonos móviles o tabletas.

Este enfoque resulta especialmente relevante en el contexto del proyecto, dado que el acceso a los *packs* de juego mediante enlaces públicos o códigos QR se realiza habitualmente desde dispositivos móviles. La correcta adaptación de la interfaz contribuye a ofrecer una experiencia de usuario consistente y fluida, independientemente del dispositivo utilizado, mejorando de forma significativa la accesibilidad y la usabilidad general del sistema.

3.5. Usabilidad y accesibilidad

El diseño de la aplicación no se ha centrado únicamente en la implementación de funcionalidades, sino también en garantizar una experiencia de uso clara, intuitiva y accesible para los distintos perfiles de usuarios. En este apartado se describen los principios de usabilidad aplicados durante el diseño del sistema, así como las consideraciones adoptadas para mejorar la accesibilidad y la validación básica de estos criterios.

3.5.1. Principios de usabilidad

Durante el diseño de la aplicación se han tenido en cuenta diversos principios de usabilidad con el objetivo de facilitar la interacción del usuario y reducir la complejidad percibida del

sistema. Estas decisiones han estado orientadas a ofrecer una experiencia de uso clara, intuitiva y adaptada a los distintos perfiles de usuarios.

Uno de los criterios fundamentales ha sido la simplicidad, priorizando interfaces limpias y fácilmente comprensibles. Para ello, se han evitado elementos visuales innecesarios y se ha organizado la información de forma jerárquica, de modo que las acciones principales resulten fácilmente identificables desde cualquier vista de la aplicación.

La navegación se ha diseñado de manera coherente y predecible, manteniendo una estructura homogénea en todas las secciones del sistema. Esta consistencia permite al usuario reconocer patrones de uso y localizar rápidamente las funcionalidades disponibles, reduciendo la curva de aprendizaje y evitando situaciones de desorientación.

Asimismo, se ha buscado minimizar el número de pasos necesarios para realizar acciones clave, como acceder a un *pack*, iniciar una partida o gestionar contenido propio. La posibilidad de jugar mediante enlaces públicos o códigos QR permite una interacción inmediata, reduciendo las barreras de entrada y mejorando la experiencia general de uso.

3.5.2. Criterios de accesibilidad

Durante el diseño de la interfaz se han tenido en cuenta diversos criterios de accesibilidad con el objetivo de facilitar el uso de la aplicación por parte del mayor número posible de usuarios, independientemente de sus características o del dispositivo utilizado. Estas consideraciones se han integrado de forma transversal en el diseño visual y estructural del sistema.

Uno de los aspectos fundamentales ha sido el uso de contrastes adecuados entre el texto y el fondo, con el fin de garantizar una correcta legibilidad y reducir la fatiga visual, especialmente durante sesiones de uso prolongadas o en entornos con condiciones de iluminación variables.

Asimismo, se ha seleccionado una tipografía clara y un tamaño de texto apropiado, favoreciendo la lectura del contenido tanto en dispositivos de escritorio como en dispositivos móviles. Los elementos interactivos, como botones y enlaces, se presentan de forma claramente identificable, evitando depender exclusivamente del color para transmitir información relevante y mejorando así la comprensión de la interfaz.

Por último, el diseño adaptable de la aplicación contribuye de manera directa a su accesibilidad, permitiendo un uso cómodo desde distintos tamaños de pantalla. Este aspecto resulta especialmente relevante en el contexto del proyecto, ya que el acceso a la aplicación mediante enlaces públicos o códigos QR se realiza habitualmente desde dispositivos móviles.

3.5.3. Validación de accesibilidad

La validación de los criterios de accesibilidad se ha llevado a cabo de forma práctica a lo largo del desarrollo del proyecto, mediante la comprobación del correcto funcionamiento de la interfaz en distintos dispositivos y tamaños de pantalla. Estas pruebas han permitido verificar

que los contenidos principales resultan visibles y accesibles sin necesidad de desplazamientos excesivos ni interacciones complejas por parte del usuario.

Asimismo, se ha prestado especial atención a la claridad de los textos, a la disposición de los elementos interactivos y a la facilidad para ejecutar las acciones principales del sistema. De este modo, se ha comprobado que las funcionalidades esenciales pueden utilizarse de forma intuitiva tanto en dispositivos de escritorio como en dispositivos móviles.

Aunque no se ha realizado una evaluación formal basada en estándares específicos de accesibilidad, como las pautas *WCAG*, las decisiones de diseño adoptadas buscan alinearse con buenas prácticas generales de accesibilidad web. Este enfoque resulta adecuado al alcance del proyecto y permite garantizar una experiencia de uso razonablemente accesible para un amplio conjunto de usuarios.

Capítulo 4

Implementación

Una vez definido el análisis y diseño del sistema, en este capítulo se aborda su implementación técnica. Se detalla la estructura del proyecto *Django* y la organización de sus distintos componentes, así como la forma en que las funcionalidades planteadas se han traducido en una solución operativa.

A lo largo del capítulo se presentan las decisiones técnicas adoptadas en cada módulo del sistema. Asimismo, se examinan los principales retos surgidos durante el desarrollo, junto con las soluciones aplicadas para garantizar la coherencia, seguridad y estabilidad de la aplicación.

De este modo, se pone de manifiesto la correspondencia entre los modelos y la arquitectura definidos previamente y su materialización práctica en el funcionamiento real del sistema.

4.1. Estructura del proyecto *Django*

El sistema se ha desarrollado utilizando el *framework Django*[2], organizado en un único proyecto principal que contiene una aplicación denominada `verdadoreto_app`. Esta decisión responde a la dimensión y alcance del proyecto, permitiendo mantener una arquitectura sencilla y centralizada sin comprometer la modularidad interna del sistema.

La estructura del proyecto sigue la organización estándar propuesta por *Django*, diferenciando claramente entre el núcleo de configuración del proyecto y la aplicación funcional. El directorio principal incluye los archivos de configuración global (`settings.py`, `urls.py`, `asgi.py`, `wsgi.py`) y el *script* de gestión `manage.py`, mientras que la aplicación contiene los módulos responsables de la lógica específica del sistema.

En la Figura 4.1 se muestra la organización general del directorio del proyecto.

```
tfg-verdadoreto/
|-- static/
|   |-- css/
|   |-- images/
|-- verdadoreto_project/
|   |-- __init__.py
|   |-- settings.py
|   |-- asgi.py
|   |-- wsgi.py
|   |-- urls.py
|-- verdadoreto_app/
|   |-- migrations/
|   |-- templates/
|   |   |-- base.html
|   |   |-- ...
|   |-- __init__.py
|   |-- admin.py
|   |-- apps.py
|   |-- consumers.py
|   |-- email_backend.py
|   |-- forms.py
|   |-- jitsi.py
|   |-- models.py
|   |-- urls.py
|   |-- views.py
|   |-- ...
|-- manage.py
|-- requirements.txt
```

Figura 4.1: Estructura general del proyecto *Django*.

Dentro de la aplicación, la organización se estructura en distintos módulos especializados:

- **models.py**: define el modelo de datos y las entidades persistentes del sistema (usuarios, *packs*, acciones, colaboraciones y salas).
- **views.py**: implementa la lógica de aplicación, gestionando las peticiones HTTP y coordinando la interacción entre modelos y plantillas.
- **forms.py**: encapsula la validación y gestión de formularios asociados a la creación y edición de contenido.
- **urls.py**: define el sistema de enrutado interno de la aplicación, asociando rutas a vistas concretas.

- **templates/**: contiene las plantillas *HTML*[8] organizadas mediante herencia.
- **admin.py**: permite la gestión administrativa de las entidades del sistema a través del panel de administración de *Django*.
- **consumers.py**: implementa la gestión de comunicaciones en tiempo real mediante *Django Channels*.
- **jitsi.py**: encapsula la integración con el servicio externo de videocomunicación.
- **email_backend.py**: implementa el *backend* personalizado de envío de correo electrónico.

Los recursos estáticos (`static/`) se encuentran organizados en el directorio principal del proyecto, separados de la lógica de aplicación, incluyendo hojas de estilo y recursos gráficos.

La aplicación sigue el patrón *MVT* (Modelo–Vista–Plantilla)[6], propio de *Django*. Este enfoque permite separar claramente la lógica de negocio, la representación de los datos y la interfaz de usuario, favoreciendo la mantenibilidad y escalabilidad del sistema.

El sistema de enrutado distingue entre rutas públicas, accesibles sin autenticación (acceso mediante enlaces o códigos QR[17]), y rutas privadas, protegidas mediante mecanismos de autenticación y autorización. Esta separación facilita la aplicación consistente de controles de acceso y refuerza la seguridad del sistema.

Finalmente, el proyecto incorpora dependencias externas especificadas en el archivo `requirements.txt`, lo que garantiza la reproducibilidad del entorno de ejecución y facilita su despliegue en distintos entornos.

4.2. Gestión de usuarios y autenticación

4.2.1. Sistema de autenticación base

La gestión de usuarios constituye uno de los pilares fundamentales del sistema, ya que determina el acceso a las distintas funcionalidades de la aplicación y garantiza la seguridad de los datos.

La aplicación se apoya en el sistema de autenticación integrado en *Django*, que proporciona un modelo de usuario robusto y mecanismos seguros para la validación de credenciales y la gestión de sesiones. Las contraseñas no se almacenan en texto plano, sino que se guarda únicamente su valor *hash*, generado mediante funciones criptográficas seguras. De este modo, incluso en el caso de acceso indebido a la base de datos, no sería posible recuperar directamente las contraseñas originales de los usuarios.

Las funcionalidades de registro e inicio de sesión se implementan mediante plantillas personalizadas, manteniendo coherencia con el diseño visual de la aplicación, pero utilizando los formularios y mecanismos estándar del *framework* para la validación y autenticación.

El sistema distingue entre distintos perfiles de acceso:

- **Usuarios invitados**: pueden acceder a funcionalidades públicas, como la visualización de *packs* compartidos y la participación en salas *online* mediante enlaces públicos o códigos

QR.

- **Usuarios registrados**: tras autenticarse, pueden crear, editar y gestionar sus propios *packs*, colaborar en contenidos compartidos y generar salas *online*.
- **Administradores**: gestionan el sistema a través del panel de administración de *Django*, con acceso completo a las entidades del modelo de datos.

El control de acceso a las vistas privadas se realiza mediante el uso del decorador `@login_required`, que restringe el acceso a usuarios autenticados y redirige automáticamente al formulario de inicio de sesión cuando es necesario. No obstante, la autenticación por sí sola no resulta suficiente en operaciones sensibles.

Por ello, el sistema implementa un control de autorización a nivel de recurso. En la gestión de *packs*, por ejemplo, se verifica que el usuario autenticado sea el propietario del contenido o un colaborador con permisos suficientes antes de permitir su modificación. Esta lógica se centraliza en funciones auxiliares de comprobación, evitando duplicación de código y garantizando coherencia en todas las vistas.

Las sesiones de usuario se gestionan mediante el sistema de sesiones de *Django*, que permite mantener el estado de autenticación entre peticiones *HTTP*, asegurando una experiencia de uso continua y segura.

Además, los formularios de autenticación y registro incorporan protección frente a ataques *CSRF*[14] mediante los mecanismos automáticos proporcionados por *Django*, reforzando la seguridad de las interacciones del usuario.

En conjunto, el sistema de gestión de usuarios integra autenticación, autorización y control de sesiones, estableciendo una base sólida de seguridad que permite diferenciar claramente entre acceso público y funcionalidades restringidas.

4.2.2. Implementación del sistema de recuperación de contraseña

Con el fin de mejorar la seguridad y usabilidad del sistema, se ha incorporado un mecanismo de recuperación de contraseña que permite a los usuarios restablecer sus credenciales en caso de olvido, sin intervención manual por parte del administrador.

Flujo funcional del proceso

El proceso de recuperación de contraseña sigue el flujo estándar recomendado en aplicaciones web seguras:

1. El usuario accede al formulario de recuperación e introduce la dirección de correo electrónico asociada a su cuenta.
2. El sistema procesa la solicitud y, si la dirección corresponde a una cuenta válida, genera el mecanismo de restablecimiento sin revelar información adicional al usuario.
3. Se genera un *token* criptográficamente seguro y de validez limitada, utilizando el sistema de generación de *tokens* incorporado en *Django*.

4. Se construye un enlace único que incorpora el identificador codificado del usuario y el *token* generado.
5. El enlace es enviado al correo electrónico del usuario.
6. El usuario accede al enlace y puede establecer una nueva contraseña tras validarse correctamente el *token*.

Este mecanismo evita la transmisión de contraseñas en texto plano y garantiza que únicamente el propietario de la cuenta, con acceso al correo electrónico asociado, pueda completar el proceso de restablecimiento.

Personalización de vistas y plantillas

La implementación se apoya en las vistas de autenticación estándar proporcionadas por *Django*, integrándolas en el sistema de enrutado del proyecto y configurándolas para utilizar plantillas personalizadas. Dichas plantillas se ubican en el directorio `templates/registration/`, siguiendo la convención del *framework*, lo que permite mantener coherencia con el diseño visual general de la aplicación.

Al apoyarse en la lógica interna de generación y validación de *tokens* proporcionada por *Django*, se evita la reimplementación manual de mecanismos criptográficos, reduciendo el riesgo de introducir vulnerabilidades.

Integración con *SendGrid*

Para posibilitar el envío real de correos electrónicos en producción, se ha integrado el servicio externo *SendGrid*[21][22] como proveedor de correo transaccional. La elección de esta plataforma responde a la necesidad de disponer de un mecanismo fiable y escalable de entrega de correos electrónicos desde un entorno desplegado en una plataforma *PaaS*, sin necesidad de gestionar infraestructura *SMTP* propia.

El envío se realiza mediante la *API* web de *SendGrid*, accediendo a ella mediante peticiones *HTTPS*. Esta comunicación se autentica a través de una clave privada (*API Key*), almacenada como variable de entorno en el servidor de despliegue *Render*. De este modo, las credenciales no se incluyen en el código fuente ni en el repositorio del proyecto.

Configuración del *backend* de correo

Para desacoplar la lógica de aplicación del proveedor específico, se ha configurado un *backend* de correo personalizado dentro del proyecto. Este *backend* encapsula la comunicación con la *API* de *SendGrid*, recibiendo los mensajes generados por *Django* y gestionando su envío externo.

La configuración del sistema de correo se realiza en el archivo `settings.py`, donde se define el `EMAIL_BACKEND` correspondiente y se leen las variables de entorno necesarias, como la clave de acceso y la dirección de remitente por defecto. Esta separación entre configuración y lógica garantiza mayor seguridad y facilita la portabilidad entre entornos de desarrollo y producción.

Seguridad y gestión de errores

Desde el punto de vista de la seguridad, el sistema no revela si una dirección de correo electrónico está registrada o no, evitando la enumeración de usuarios. El *token* generado para el restablecimiento tiene un tiempo de validez limitado y se invalida automáticamente tras su uso.

En cuanto a la robustez del sistema, se incorporan mecanismos de captura de excepciones durante la comunicación con la *API* de *SendGrid*. En caso de error en el envío, la aplicación registra la incidencia mediante el sistema de *logging*, evitando que fallos externos interrumpan el funcionamiento general de la aplicación.

En conjunto, la integración de *SendGrid* completa el sistema de recuperación de contraseña, permitiendo un proceso seguro, desacoplado y coherente con la arquitectura general de la aplicación.

4.3. Implementación de *packs* y acciones

La gestión de *packs* y acciones constituye el núcleo funcional de la aplicación, ya que define el contenido jugable del sistema. Su implementación se basa directamente en el modelo de datos descrito en el capítulo anterior, trasladando las entidades conceptuales al modelo relacional gestionado por *Django*.

Modelo de datos

Los *packs* y las acciones se implementan mediante modelos definidos en el archivo `models.py`.

El modelo `Pack` representa un conjunto de contenido creado por un usuario e incluye atributos como el nombre, la categoría, el nivel y la fecha de creación y un identificador público basado en un *token* único. Este modelo incorpora una relación de clave foránea hacia el usuario creador, estableciendo la asociación uno a muchos entre usuario y *pack*.

Para permitir el acceso público sin exponer identificadores internos de base de datos, cada *pack* dispone de un *token* generado automáticamente en el momento de su creación. Dicho *token* se utiliza en la construcción de enlaces compartibles y códigos QR, y puede regenerarse cuando sea necesario, invalidando enlaces previamente distribuidos y reforzando el control sobre el acceso público al contenido.

Por su parte, el modelo `Action` (Pregunta/Reto) representa los elementos individuales que componen un *pack*. Cada acción está vinculada mediante una clave foránea al `Pack` correspondiente, utilizando eliminación en cascada (`on_delete=CASCADE`), lo que garantiza la integridad referencial en caso de borrado del *pack* asociado.

Con el objetivo de simplificar el diseño y evitar duplicación estructural, las preguntas y retos se modelan mediante esta entidad `Action`, diferenciando su comportamiento mediante un campo que indica el tipo de acción. Esta decisión permite reutilizar la misma estructura de datos y lógica

de gestión para ambos elementos, facilitando futuras ampliaciones del sistema y manteniendo coherencia en la implementación.

Asimismo, el modelo incluye un indicador booleano que determina si la acción se encuentra activa, permitiendo gestionar dinámicamente el contenido disponible en cada momento.

Lógica de aplicación

La creación, edición y eliminación de *packs* y acciones se gestiona a través de vistas específicas que implementan la lógica de negocio correspondiente.

En el caso de los *packs*, el sistema permite:

- Crear nuevos *packs* asociados automáticamente al usuario autenticado.
- Editar únicamente aquellos *packs* sobre los que el usuario tenga permisos suficientes.
- Eliminar *packs*, lo que implica la eliminación automática de las acciones asociadas.
- Regenerar el *token* público cuando se desea invalidar enlaces previamente compartidos.

Para las acciones, el sistema permite su gestión individual dentro del contexto de un *pack*, garantizando que no puedan existir acciones sin asociación válida gracias a la restricción impuesta por la clave foránea en el modelo de datos. La validación se realiza tanto a nivel de formulario como en la lógica de las vistas, reforzando la coherencia del sistema.

Durante las operaciones de creación, edición o eliminación, el sistema proporciona retroalimentación al usuario mediante el sistema de mensajes de *Django*, informando del resultado de cada acción y mejorando la experiencia de uso.

Validación y control de integridad

La validación de datos se implementa mediante formularios definidos en `forms.py`, que permiten controlar la coherencia de los datos introducidos por el usuario antes de su persistencia en la base de datos.

Además, únicamente se exponen en la interfaz los campos estrictamente necesarios, asignando el resto de atributos sensibles (como el usuario creador o el *token* público) desde la lógica del servidor.

Se incorporan comprobaciones adicionales en las vistas para asegurar que:

- El usuario autenticado tenga permiso para modificar el recurso.
- Las acciones añadidas pertenezcan realmente al *pack* seleccionado.
- No se puedan realizar operaciones inconsistentes mediante manipulación manual de URLs.

Este enfoque combinado (validación en formulario + validación en vista) refuerza la seguridad y evita estados inconsistentes en el sistema.

Organización y reutilización del contenido

El diseño adoptado permite que los *packs* actúen como unidades reutilizables de contenido. Al separar claramente *packs* y acciones, el sistema facilita la edición modular del juego y permite su reutilización en diferentes sesiones o salas *online* sin duplicar información.

En conjunto, la implementación de *packs* y acciones refleja de forma directa el modelo conceptual definido en el análisis, manteniendo coherencia entre el diseño teórico y la materialización práctica en el sistema.

4.4. Sistema de colaboración y permisos

Una de las funcionalidades diferenciales del sistema es la posibilidad de colaboración en la creación de *packs*, permitiendo que varios usuarios participen en la edición de un mismo contenido. Para ello, se mantiene un control preciso sobre los permisos de acceso mediante un mecanismo de autorización a nivel de recurso.

Esta funcionalidad se apoya en la entidad asociativa `Colaboracion`, que modela la relación muchos a muchos entre usuarios y *packs*.

Modelo de colaboración

La entidad `Colaboracion`, definida en `models.py`, actúa como tabla asociativa entre el modelo `User` y el modelo *`Pack`*. Cada instancia de colaboración establece que un usuario determinado dispone de permisos sobre un *pack* concreto.

Este modelo incluye referencias de clave foránea tanto al usuario colaborador como al *pack*, así como campos adicionales que permiten gestionar el rol asignado y la fecha de creación de la colaboración. Asimismo, se incorpora una referencia al usuario que concede el permiso, lo que permite mantener trazabilidad sobre la asignación de accesos.

Para evitar duplicidades y garantizar la consistencia del sistema, se define una restricción de unicidad sobre la combinación usuario–*pack*, impidiendo que un mismo usuario pueda ser añadido varias veces como colaborador del mismo recurso. Esta restricción se implementa mediante una *UniqueConstraint* definida en el modelo.

Este enfoque proporciona mayor flexibilidad que una relación *ManyToManyField* simple, ya que permite almacenar información adicional asociada a cada relación y gestionar permisos de forma más granular.

Sistema de roles

El modelo de colaboración distingue dos roles principales:

- **Lector**: permite acceder al *pack* con fines de visualización y juego, sin capacidad de modificación del contenido.

- **Editor**: habilita la creación, edición y eliminación de acciones, así como la modificación de los atributos del *pack*.

Determinadas operaciones críticas, como la eliminación completa del *pack* o la gestión de colaboradores, permanecen restringidas exclusivamente al propietario del recurso.

Esta diferenciación permite implementar un control de acceso más preciso y adaptable a distintos escenarios de colaboración.

Gestión de permisos

La autorización en el sistema no se limita a comprobar si un usuario está autenticado, sino que se realiza a nivel de recurso. En la gestión de *packs*, antes de permitir operaciones como edición o eliminación, el sistema verifica si el usuario es:

- El propietario del *pack*, o
- Un colaborador con rol *editor* registrado en la entidad `Colaboracion`.

Estas comprobaciones se centralizan en funciones auxiliares que encapsulan la lógica de autorización, evitando duplicación de código en distintas vistas y garantizando coherencia en todo el sistema.

Control de integridad y seguridad

Para evitar accesos indebidos, el sistema incorpora validaciones adicionales que impiden:

- Que un usuario no autorizado modifique un *pack* manipulando manualmente la URL.
- Que un colaborador edite un recurso distinto al que tiene asignado.
- Que se generen colaboraciones duplicadas entre un mismo usuario y *pack*.

Estas restricciones se implementan tanto a nivel de modelo (mediante restricciones de unicidad cuando procede) como en la lógica de las vistas.

En conjunto, el sistema de colaboración refuerza el modelo de permisos de la aplicación, permitiendo edición compartida de contenido sin comprometer la seguridad ni la coherencia de los datos.

4.5. Generación de enlaces públicos y códigos QR

Con el objetivo de facilitar el acceso a los *packs* de juego sin necesidad de registro, el sistema implementa un mecanismo de enlaces públicos basados en identificadores únicos. Cada *pack* dispone de un *token* público que forma parte de la URL y permite acceder directamente a la vista pública del juego, evitando el uso de identificadores internos de la base de datos.

Este *token* se genera automáticamente en el momento de creación del *pack* y se almacena como un campo propio del modelo. Su carácter opaco y no predecible incrementa la seguridad del sistema, dificultando el acceso no autorizado mediante enumeración de recursos. Además,

el sistema permite regenerar dicho *token* cuando sea necesario, invalidando enlaces públicos anteriores y proporcionando un mayor control al propietario del *pack*.

Las vistas asociadas a estos enlaces se encuentran diferenciadas de las vistas privadas del sistema. Aunque el acceso al contenido no requiere autenticación, las operaciones de modificación permanecen restringidas a usuarios con permisos adecuados.

A partir del enlace público asociado al *token*, la aplicación ofrece la posibilidad de generar un código QR que facilita el acceso desde dispositivos móviles. El código QR se genera dinámicamente en memoria cuando el usuario autorizado lo solicita, sin necesidad de almacenar archivos en el servidor. Para ello, se construye la URL pública completa del *pack* y se codifica en formato QR, devolviendo la imagen resultante como un archivo descargable.

El acceso a la generación de enlaces y códigos QR está restringido a usuarios con permisos de edición sobre el *pack*, garantizando que únicamente el propietario o los colaboradores autorizados puedan compartir el contenido.

Este mecanismo resulta especialmente útil en contextos sociales y presenciales, permitiendo iniciar una partida de forma rápida mediante el escaneo del código QR o el uso directo del enlace.

4.6. Vista pública del juego

La aplicación muestra una vista pública del juego que permite acceder a un *pack* sin necesidad de registro ni autenticación previa. Esta vista se activa cuando el acceso se realiza mediante un enlace público o un código QR asociado al *token* del *pack*, facilitando una experiencia de uso inmediata y reduciendo las barreras de entrada para nuevos usuarios.

En esta vista, el sistema recupera el *pack* correspondiente al *token* proporcionado en la URL mediante una consulta filtrada a la base de datos a través del ORM de *Django*. Si el *token* no existe o ha sido invalidado, se devuelve una respuesta controlada, evitando la exposición de información interna.

Una vez localizado el recurso, se presenta únicamente la información necesaria para el desarrollo del juego. El usuario puede participar en la partida y visualizar las acciones del *pack*, pero no dispone de opciones de edición ni de gestión del contenido. De este modo, se garantiza que el acceso público se limite a un uso de solo lectura, preservando la integridad de los datos almacenados.

La vista pública está diseñada para ser clara y sencilla, priorizando la experiencia de juego frente a la complejidad funcional. Este enfoque resulta especialmente adecuado para contextos sociales o presenciales, en los que varios participantes acceden desde dispositivos móviles tras escanear un código QR o recibir un enlace compartido.

Desde el punto de vista de seguridad, el acceso público se apoya exclusivamente en el *token* del *pack*, sin exponer identificadores internos ni requerir credenciales. La separación entre vistas públicas y privadas permite aplicar controles de acceso estrictos en las funcionalidades de edición, mientras que se mantiene un acceso abierto y controlado para la participación en el juego.

4.7. Implementación de salas *online*

Con el objetivo de ampliar la experiencia de juego más allá del entorno local y permitir la participación remota de varios usuarios, la aplicación incorpora la funcionalidad de salas *online*. Esta característica amplía el alcance del sistema, posibilitando que varios usuarios puedan jugar simultáneamente a un mismo *pack* desde distintos dispositivos y ubicaciones.

Cada sala *online* se asocia a un *pack* concreto y es creada por un usuario con los permisos adecuados. El sistema gestiona la información necesaria para identificar la sala y controlar su acceso, manteniendo coherencia con el modelo de datos y el sistema de permisos definidos previamente.

La implementación de esta funcionalidad combina la gestión interna de las salas dentro de la aplicación con la integración de mecanismos de comunicación en tiempo real. En los apartados siguientes se describen las principales decisiones técnicas adoptadas y la forma en que se ha materializado esta funcionalidad en el sistema.

4.7.1. Modelo de datos de la sala

La funcionalidad de salas *online* se implementa mediante el modelo `VideoRoom`, definido en el archivo `models.py`. Este modelo representa una sesión de juego activa asociada a un *pack* concreto y almacena la información necesaria para su identificación y gestión dentro del sistema.

Cada sala mantiene una relación de clave foránea con el modelo `Pack`, garantizando que toda sesión de juego esté vinculada a un contenido válido. Asimismo, se establece una relación con el usuario creador de la sala, que actúa como anfitrión y dispone de permisos especiales durante la sesión.

Entre los campos principales del modelo se incluyen:

- La referencia al *pack* asociado y al usuario anfitrión.
- Un identificador único o código de sala, utilizado para construir el enlace de acceso.
- El estado de la sala, que permite distinguir entre distintas fases de la sesión.
- Las marcas temporales de creación y actualización.
- Un campo de expiración que limita la validez temporal de la sala.

El modelo incorpora además un mecanismo de control temporal mediante un campo de expiración, que permite limitar la validez de las salas y facilitar su eliminación automática tras un periodo de inactividad.

Este diseño mantiene la coherencia con el modelo entidad–relación definido previamente, trasladando la entidad conceptual Sala *online* a una implementación concreta dentro del entorno *Django*.

4.7.2. Gestión del ciclo de vida de la sala

Además de su definición estructural, las salas *online* incorporan un ciclo de vida gestionado dinámicamente por la aplicación. Este ciclo permite controlar la creación, activación, mantenimiento y finalización de cada sesión, garantizando un uso eficiente de los recursos del sistema.

Creación de la sala

La creación de una sala se realiza desde una vista protegida, accesible únicamente a usuarios autenticados con permisos sobre el *pack* seleccionado. En el momento de la creación se genera un código único de sala, se establece el estado inicial como *lobby* y se asocia el usuario creador como anfitrión.

Asimismo, se inicia un mecanismo de control temporal que define el tiempo máximo de validez de la sala, almacenando una fecha de expiración en el campo correspondiente.

Estados de la sesión

El modelo incorpora un campo de estado que permite reflejar la fase actual de la sala. Inicialmente, la sala se encuentra en estado *lobby*, lo que indica que está disponible y esperando participantes. Durante la partida puede pasar a un estado activo, permitiendo al *host* llevar la dinámica de la partida y finalmente a un estado finalizado cuando la sesión concluye.

Este sistema de estados permite gestionar el comportamiento de la interfaz y restringir determinadas operaciones según el momento de la sesión.

Control temporal y expiración

Las salas tienen un carácter temporal. Para evitar la acumulación de sesiones inactivas, el sistema implementa un mecanismo de control basado en un campo de expiración (`expires_at`).

En el momento de la creación se establece un tiempo de vida inicial. Durante la actividad de la sala, dicho tiempo puede renovarse dinámicamente, extendiendo su validez. Si la sala permanece inactiva y supera el tiempo definido, se considera expirada.

Además, el sistema dispone de un mecanismo de limpieza que elimina automáticamente las salas cuyo tiempo de expiración ha sido superado, manteniendo así la integridad y eficiencia del entorno.

Finalización de la sala

La sala puede finalizarse explícitamente por el anfitrión o de forma automática cuando expira su tiempo de vida. En ambos casos, el estado se actualiza y se impide el acceso posterior a la sesión.

Este enfoque permite modelar las salas como entidades dinámicas y controladas, alineadas con el carácter temporal de una partida en tiempo real y evitando la persistencia indefinida de recursos innecesarios.

4.7.3. Comunicación en tiempo real (*Django Channels*)

La implementación de salas *online* requiere, además del flujo tradicional basado en peticiones HTTP, un mecanismo de comunicación bidireccional que permita intercambiar información de forma inmediata entre cliente y servidor. El modelo de petición–respuesta de HTTP resulta adecuado para la navegación y gestión de recursos, pero no para escenarios donde se necesita actualizar el estado de una sesión en tiempo real o notificar cambios a múltiples participantes simultáneamente.

Para cubrir esta necesidad, el sistema incorpora *Django Channels*, que extiende las capacidades de *Django* permitiendo gestionar conexiones *WebSocket* dentro del mismo proyecto. Mediante *WebSockets*, el navegador puede mantener una conexión persistente con el servidor, lo que facilita el envío de eventos y actualizaciones sin necesidad de realizar consultas periódicas.

A nivel de arquitectura, esta integración implica el uso de un entorno *ASGI (Asynchronous Server Gateway Interface)*, en lugar del enfoque tradicional basado únicamente en *WSGI*. De este modo, el proyecto puede atender tanto peticiones HTTP convencionales como conexiones *WebSocket* de forma compatible.

La lógica de comunicación en tiempo real se implementa a través de *consumers*, definidos en el archivo `consumers.py`. Estos componentes gestionan los eventos asociados a cada sala, permitiendo, entre otras funciones, notificar la conexión o desconexión de participantes y sincronizar información de estado durante la sesión. Para organizar la comunicación entre múltiples clientes conectados a una misma sala, se emplea un sistema de grupos asociado al identificador de la sala, de forma que los mensajes enviados se distribuyen a todos los participantes correspondientes.

Con esta aproximación, la aplicación mantiene la coherencia del estado de la sala en escenarios multiusuario y proporciona una base sólida para funcionalidades en tiempo real, complementando la integración de videocomunicación con una capa propia de coordinación y sincronización dentro del sistema.

4.7.4. Integración con *Jitsi*

La comunicación audiovisual de las salas *online* se implementa mediante la integración de *Jitsi as a Service (JaaS)*, proporcionado por *8x8*[23]. Esta solución permite incorporar videollamadas en tiempo real sin necesidad de gestionar infraestructura propia para transmisión de audio y vídeo.

La integración se realiza a través de la *Jitsi Meet External API*, cargada dinámicamente en la plantilla de la sala. En el lado del cliente se instancia un objeto `JitsiMeetExternalAPI`,

indicando el dominio del servicio, el nombre de la sala y las opciones de configuración necesarias. La videollamada se establece directamente entre el navegador del usuario y los servidores de *Jitsi* mediante tecnologías *WebRTC*, de modo que el servidor *Django* no interviene en la transmisión multimedia.

Generación de credenciales mediante JWT

Para controlar el acceso a las salas y habilitar permisos diferenciados (por ejemplo, anfitrión con privilegios de moderador), el sistema genera un *JSON Web Token (JWT)* firmado en el servidor. Este *token* incluye, entre otros, los siguientes elementos:

- El identificador de la aplicación (`sub`), correspondiente al *App ID* configurado en *JaaS*.
- El identificador lógico de la sala (`room`), asociado al código interno almacenado en el modelo `VideoRoom`.
- La información del usuario (identificador, nombre visible y rol de moderador cuando procede).
- Un tiempo de expiración limitado para reforzar la seguridad.

El *token* se firma utilizando el algoritmo `RS256` con una clave privada configurada en el entorno del servidor. Posteriormente, se envía al cliente como parte del contexto de la plantilla y se utiliza al instanciar la *API* de *Jitsi*, permitiendo autenticar la sesión sin exponer credenciales sensibles.

Construcción del nombre de la sala

En el cliente, el nombre completo de la sala se construye combinando el identificador de la aplicación con el código único de la sala, siguiendo el formato requerido por JaaS. De este modo, cada sesión queda correctamente aislada dentro del espacio de nombres asociado a la aplicación.

Separación de responsabilidades

La arquitectura adoptada establece una separación clara entre:

- La gestión lógica de la sala, controlada por *Django* (creación, permisos, estado y expiración).
- La sincronización de eventos del juego, gestionada mediante *Django Channels*.
- La comunicación audiovisual, gestionada directamente por Jitsi.

Este enfoque permite descargar al servidor de la aplicación de la carga asociada a la transmisión multimedia, manteniendo al mismo tiempo el control sobre el acceso y la coordinación de la sesión de juego. Además, la inclusión del campo `provider` en el modelo facilita la posibilidad de sustituir el servicio de videocomunicación en el futuro sin alterar la lógica interna del sistema.

4.8. Gestión de errores y *logging*

La aplicación incorpora mecanismos de gestión de errores y retroalimentación al usuario con el objetivo de mejorar la experiencia de uso y garantizar la robustez del sistema.

Durante la interacción con la aplicación, las operaciones principales —como la creación, modificación o eliminación de *packs*, acciones o salas— generan mensajes informativos mediante el sistema de mensajes de *Django*. Estos mensajes permiten al usuario conocer de forma clara si una operación se ha completado correctamente o si se ha producido algún error, favoreciendo una interacción más comprensible y controlada.

En el lado del servidor, se implementan comprobaciones explícitas de permisos y validaciones adicionales antes de ejecutar operaciones sensibles. Para el acceso a recursos identificados por parámetros dinámicos (como códigos de sala o *tokens* públicos), se emplean funciones como `get_object_or_404`, que permiten manejar de forma controlada la ausencia de recursos y evitar la exposición de información interna. De este modo, el sistema responde de forma coherente ante accesos indebidos o referencias inválidas.

En el contexto de las salas *online*, se controla el estado y la expiración de las sesiones antes de permitir su uso. Asimismo, en la comunicación en tiempo real mediante WebSocket se contemplan los eventos de conexión, cierre y error, permitiendo detectar incidencias y mantener la estabilidad de la sesión incluso ante desconexiones inesperadas.

Durante el desarrollo del proyecto se ha utilizado el sistema de *logging* integrado en *Django* como herramienta de depuración, registrando eventos relevantes para analizar el flujo de ejecución y localizar posibles errores. Aunque no se ha implementado un sistema avanzado de monitorización en producción, la estructura actual permite ampliar fácilmente esta funcionalidad en el futuro, incorporando un sistema de registro más detallado orientado al análisis de incidencias y comportamiento del sistema.

En conjunto, estos mecanismos contribuyen a reforzar la fiabilidad de la aplicación y facilitan su mantenimiento y evolución.

Capítulo 5

Integración y despliegue

Una vez desarrolladas las distintas funcionalidades del sistema, resulta necesario abordar el proceso mediante el cual la aplicación pasa del entorno de desarrollo local a un entorno de producción accesible para los usuarios finales. Este capítulo describe las decisiones técnicas y configuraciones adoptadas para garantizar un despliegue estable, seguro y reproducible.

La integración y el despliegue constituyen una fase crítica en el ciclo de vida de una aplicación web, ya que implican la adaptación del sistema a un entorno real de ejecución, donde intervienen factores como la gestión de dependencias, la configuración de variables de entorno, la conexión a bases de datos externas, el tratamiento de archivos estáticos y la configuración de dominios y certificados de seguridad.

En los apartados siguientes se detallan las diferencias entre el entorno de desarrollo y el entorno de producción, la configuración específica adoptada para el despliegue en la nube mediante una plataforma *PaaS*[19], la gestión de la base de datos en producción y los mecanismos empleados para garantizar la disponibilidad y el mantenimiento del sistema.

5.1. Entorno de desarrollo

El desarrollo de la aplicación se ha llevado a cabo en un entorno local controlado, configurado específicamente para facilitar la implementación, prueba y depuración del sistema antes de su despliegue en producción.

Para aislar las dependencias del proyecto y garantizar la reproducibilidad del entorno, se ha utilizado un entorno virtual de *Python*[24]. Este mecanismo permite instalar las bibliotecas necesarias sin afectar a la configuración global del sistema operativo. Todas las dependencias utilizadas se encuentran especificadas en el archivo `requirements.txt`, lo que facilita la recreación del entorno en otros equipos o servidores.

Durante el desarrollo, la aplicación se ha ejecutado utilizando el servidor integrado de *Django*[2], adecuado para pruebas locales pero no recomendado para entornos de producción. En esta fase se ha trabajado con la configuración `DEBUG = True`, lo que permite mostrar información

detallada de errores y facilita la identificación de problemas durante la implementación.

En cuanto a la persistencia de datos, se ha utilizado inicialmente una base de datos *SQLite*[15] para el desarrollo local, dada su simplicidad de configuración y su integración directa con *Django*. Esta elección permite un arranque rápido del proyecto sin necesidad de configurar servicios adicionales. No obstante, la configuración del sistema se ha diseñado desde el inicio para permitir la migración a una base de datos *PostgreSQL*[16] en el entorno de producción, garantizando así mayor robustez y escalabilidad.

Además, se ha empleado el sistema de control de versiones *Git*[25] para gestionar la evolución del código fuente a lo largo del desarrollo. El uso de control de versiones ha permitido registrar los cambios realizados, mantener un historial completo de modificaciones y facilitar la reversión a estados anteriores en caso necesario. El repositorio se ha alojado en la plataforma *GitHub*[26][27], lo que ha permitido disponer de una copia remota del proyecto y ha facilitado su integración posterior con la plataforma de despliegue en la nube.

Asimismo, se han empleado variables de entorno para almacenar información sensible, como claves privadas y configuraciones externas, evitando su inclusión directa en el código fuente. Este enfoque refuerza la seguridad del proyecto y facilita la transición entre distintos entornos de ejecución.

En conjunto, el entorno de desarrollo ha permitido implementar y validar progresivamente las funcionalidades del sistema en un contexto seguro y controlado, asegurando una base sólida antes de proceder al despliegue en la nube.

5.2. Configuración de producción

Una vez finalizada la fase de desarrollo, fue necesario adaptar la configuración del proyecto para su ejecución en un entorno de producción. Esta transición implica reforzar aspectos de seguridad, rendimiento y estabilidad que no resultan críticos durante el desarrollo local.

Aspecto	Entorno de desarrollo	Entorno de producción
Modo DEBUG	Activado (`DEBUG = True`)	Desactivado (`DEBUG = False`)
Servidor	Servidor integrado de Django	Servidor WSGI/ASGI gestionado por la plataforma
Base de datos	SQLite local	PostgreSQL gestionado externamente
Gestión de secretos	Variables en archivo local	Variables de entorno seguras
Archivos estáticos	Servidos automáticamente por Django	Recopilados mediante `collectstatic`
HTTPS	No obligatorio	Obligatorio
Control de errores	Mensajes detallados en navegador	Mensajes genéricos y logging interno

Tabla 5.1: Comparativa entre entorno de desarrollo y entorno de producción.

Como se observa en la Tabla 5.1, el entorno de producción requiere configuraciones adicionales orientadas a la seguridad, estabilidad y rendimiento del sistema.

En primer lugar, se desactiva el modo de depuración estableciendo `DEBUG = False`. Esta configuración evita la exposición de información sensible en caso de error y obliga a definir explícitamente los dominios permitidos mediante la variable `ALLOWED_HOSTS`, restringiendo así el acceso a la aplicación únicamente a los dominios autorizados.

Asimismo, se han configurado variables de entorno para almacenar información sensible, como la `SECRET_KEY` de *Django*, credenciales de bases de datos y claves privadas asociadas a servicios externos. La `SECRET_KEY` es utilizada por el *framework* para firmar datos sensibles, como *cookies* de sesión, *tokens* de autenticación y mecanismos de protección frente a ataques *CSRF*. Su confidencialidad resulta esencial para garantizar la integridad y autenticidad de la información intercambiada entre el cliente y el servidor.

En relación con el sistema de recuperación de contraseña, también se han configurado variables de entorno específicas para la integración del servicio de correo transaccional utilizado en producción. En particular, se define la variable `SENDGRID_API_KEY`, que permite autenticar las peticiones realizadas a la *API* del proveedor externo de correo, así como la variable `DEFAULT_FROM_EMAIL`, que establece la dirección de remitente utilizada en los mensajes enviados por el sistema.

La configuración del parámetro `EMAIL_BACKEND` en producción permite que los mensajes generados por el sistema de autenticación de *Django* sean redirigidos al servicio externo de envío de correo, garantizando la entrega efectiva de notificaciones asociadas al restablecimiento de contraseña sin necesidad de gestionar infraestructura *SMTP* propia.

Del mismo modo, las claves privadas empleadas para la generación de *JWT* en la integración con servicios externos (como el sistema de videollamada) se almacenan exclusivamente como variables de entorno, evitando su inclusión en el repositorio de código. Este enfoque impide la exposición accidental de credenciales y facilita la gestión independiente de configuraciones entre los entornos de desarrollo y producción.

En producción, el servidor de desarrollo de *Django* es reemplazado por un servidor compatible con el estándar *WSGI/ASGI*, gestionado por la plataforma de despliegue. En este proyecto, la aplicación se despliega en la plataforma *Render*, que actúa como proveedor *PaaS* y se encarga de la gestión del entorno de ejecución, la asignación de recursos y el reinicio automático del servicio en caso de fallo.

Otro aspecto relevante es la correcta configuración de los archivos estáticos, ya que en producción no son servidos automáticamente por el servidor de desarrollo de *Django*. Para ello, se define el parámetro `STATIC_ROOT` y se ejecuta el comando `collectstatic`, asegurando que los recursos *CSS*[9], *JavaScript*[12] e imágenes estén disponibles de forma eficiente.

En cuanto a la base de datos, la configuración se adapta para utilizar un sistema gestor externo, optimizado para entornos productivos. En este proyecto se emplea *PostgreSQL* en producción, gestionado por la propia plataforma *Render*. Esta configuración se realiza mediante variables de entorno que definen el motor, el nombre de la base de datos y las credenciales de acceso.

Finalmente, se habilita el uso obligatorio de *HTTPS* en producción, garantizando que todas las comunicaciones entre el cliente y el servidor se realicen de forma cifrada. Esta medida resulta esencial para proteger la autenticación de los usuarios y la transmisión de información sensible.

En conjunto, la configuración de producción se ha diseñado para ofrecer un entorno seguro, estable y alineado con las buenas prácticas recomendadas para aplicaciones web desplegadas en la nube.

5.3. Archivos estáticos y `collectstatic`

La aplicación hace uso de archivos estáticos para definir el estilo visual y la presentación de la interfaz, incluyendo hojas de estilo (*CSS*), iconos e imágenes. Durante el desarrollo en entorno local, *Django* gestiona estos recursos de forma automática, permitiendo su uso directo sin necesidad de configuraciones adicionales. Sin embargo, en un entorno de producción esta práctica no es recomendable por motivos de rendimiento y seguridad.

En producción, los archivos estáticos deben recopilarse en un único directorio mediante el comando:

```
python manage.py collectstatic
```

Este proceso copia todos los archivos definidos en los distintos directorios `static/` de las aplicaciones hacia el directorio configurado en el parámetro `STATIC_ROOT`. De este modo, los

recursos quedan centralizados y pueden ser servidos de forma eficiente por la infraestructura de la plataforma de despliegue que gestiona el entorno de ejecución.

En este proyecto, los recursos estáticos se encuentran organizados en el directorio `static/` del proyecto, separados de la lógica de aplicación. En el entorno de producción desplegado en *Render*, el proceso de recopilación de archivos estáticos se ejecuta durante la fase de construcción del servicio, garantizando que los recursos estén disponibles antes de iniciar la aplicación.

La correcta configuración de los parámetros `STATIC_URL` y `STATIC_ROOT` resulta esencial para asegurar que los archivos se sirvan correctamente en producción. Esta separación entre el código de aplicación y los recursos estáticos contribuye a mejorar el rendimiento, facilitar la escalabilidad y mantener una arquitectura alineada con las buenas prácticas recomendadas para aplicaciones web.

5.4. Base de datos en producción

Durante la fase de desarrollo, la aplicación utiliza *SQLite* como sistema gestor de base de datos, debido a su simplicidad de configuración y a que no requiere servicios externos adicionales. Sin embargo, para el entorno de producción se ha optado por un sistema gestor más robusto y adecuado para aplicaciones desplegadas en la nube.

En este proyecto, la base de datos en producción se implementa mediante *PostgreSQL*, gestionado como servicio externo por la plataforma *Render*. Esta solución permite disponer de un sistema transaccional completo, con mayor rendimiento, mejor gestión de concurrencia y soporte avanzado para integridad de datos.

La configuración de la base de datos en producción no se realiza directamente en el código fuente, sino a través de variables de entorno proporcionadas por la plataforma de despliegue. Estas variables incluyen parámetros como el motor de base de datos, el nombre, el usuario, la contraseña, el *host* y el puerto. De este modo, se evita exponer credenciales sensibles en el repositorio y se facilita la separación entre entornos.

La conexión entre la aplicación *Django* y la base de datos se define en el archivo `settings.py`, donde se adapta la configuración en función del entorno de ejecución. Esta estrategia permite mantener una única base de código para desarrollo y producción, modificando únicamente las variables de entorno según el contexto.

Asimismo, el uso de un gestor externo garantiza la persistencia de los datos incluso en caso de reinicio del servicio web, así como la posibilidad de realizar copias de seguridad y tareas de mantenimiento desde la propia plataforma.

En conjunto, la adopción de *PostgreSQL* en producción refuerza la estabilidad, escalabilidad y fiabilidad del sistema, alineándose con las buenas prácticas recomendadas para aplicaciones web desplegadas en entornos profesionales.

5.5. Despliegue en la nube

5.5.1. *Render*

El despliegue de la aplicación se ha realizado utilizando la plataforma *Render*[20], que actúa como proveedor *Platform as a Service (PaaS)*, un servicio de alojamiento en la nube que permite desplegar aplicaciones web de forma sencilla. Esta plataforma permite automatizar el proceso de construcción, configuración y ejecución de la aplicación a partir de un repositorio remoto.

La elección de esta plataforma se fundamenta tanto en criterios técnicos como económicos. Desde el punto de vista técnico, *Render* ofrece soporte nativo para aplicaciones basadas en *Python* y *Django*, integración directa con repositorios de *GitHub*, gestión sencilla de variables de entorno y compatibilidad con servidores *WSGI/ASGI*. Además, proporciona servicios gestionados de base de datos *PostgreSQL*, lo que simplifica la arquitectura del sistema y reduce la necesidad de configuración manual de infraestructura.

Desde el punto de vista económico, *Render* dispone de planes gratuitos y de coste reducido orientados a proyectos académicos y aplicaciones de pequeña o media escala. Esta característica permite mantener la aplicación desplegada en un entorno real sin necesidad de asumir costes elevados de infraestructura. No obstante, en escenarios de mayor concurrencia o uso intensivo de recursos, sería necesario optar por planes superiores con mayor capacidad de procesamiento y memoria, lo que implicaría un incremento proporcional del coste operativo.

5.5.2. Proceso de despliegue

El flujo de despliegue comienza con la integración del repositorio del proyecto alojado en *GitHub*[26][27]. Cada vez que se realiza un *push* a la rama principal, *Render* detecta automáticamente los cambios y ejecuta un nuevo proceso de construcción del servicio. En esta fase se instalan las dependencias y se recopilan los archivos estáticos, garantizando que los recursos web estén disponibles en producción:

```
pip install -r requirements.txt && \
python manage.py collectstatic --noinput
```

Tras completar la fase de construcción, la plataforma inicia el servicio ejecutando la aplicación en modo *ASGI*, lo cual resulta necesario para soportar tanto peticiones HTTP tradicionales como conexiones *WebSocket* empleadas por *Django Channels*. En el arranque se aplican las migraciones pendientes y se levanta el servidor *Daphne*, escuchando en el puerto asignado por la plataforma:

```
python manage.py migrate --noinput && \
daphne -b 0.0.0.0 -p $PORT --proxy-headers verdadoreto.asgi:application
```

Este enfoque permite mantener el esquema de la base de datos sincronizado con el código desplegado. Aunque en sistemas de gran escala las migraciones suelen gestionarse como un paso independiente dentro del proceso de despliegue, en el contexto de este proyecto resulta una solución práctica y segura, ya que si no existen migraciones pendientes el comando no realiza ningún cambio adicional en la base de datos.

El modelo de despliegue adoptado sigue un enfoque de integración continua básica, en el que cada actualización del código genera automáticamente una nueva versión desplegada. Además, Render proporciona mecanismos integrados para gestionar variables de entorno, visualizar registros (*logs*) y reiniciar el servicio en caso de fallo, facilitando el mantenimiento del sistema.

5.6. Dominio, *DNS* y HTTPS

Una vez desplegada la aplicación en el entorno de producción, se ha configurado su acceso mediante un dominio público que permite a los usuarios utilizar el sistema a través de un navegador web estándar. La asignación del dominio y su resolución mediante el sistema de nombres de dominio (*DNS*) posibilitan que la aplicación sea accesible de forma sencilla y estable desde Internet, traduciendo el nombre legible por humanos en la dirección del servidor donde se aloja el servicio.

El uso de un dominio propio resulta especialmente relevante en un sistema que ofrece acceso mediante enlaces públicos y códigos QR, ya que garantiza URLs claras, persistentes y fácilmente compartibles entre los participantes del juego. Esta configuración mejora la experiencia de usuario y refuerza la percepción de la aplicación como un servicio web completo.

Asimismo, el acceso a la aplicación se realiza mediante el protocolo *HTTPS*, que cifra la comunicación entre el cliente y el servidor utilizando *TLS*. El uso de *HTTPS* protege la información intercambiada, incluyendo credenciales de acceso y *tokens* utilizados por la aplicación, y resulta esencial para garantizar la seguridad en un entorno de producción. Además, el uso de conexiones seguras contribuye a la compatibilidad con navegadores modernos y servicios externos integrados en la aplicación. En este proyecto, el certificado digital necesario para habilitar *HTTPS* es gestionado automáticamente por la plataforma de despliegue, simplificando su configuración y renovación periódica.

La correcta configuración del dominio, la resolución *DNS* y el uso de HTTPS constituyen elementos fundamentales para el despliegue final del sistema, asegurando un acceso seguro, confiable y adecuado para un uso real por parte de los usuarios.

5.7. Monitorización y mantenimiento

Una vez desplegada la aplicación en producción, resulta fundamental disponer de mecanismos que permitan supervisar su funcionamiento y garantizar su estabilidad a lo largo del tiempo. La

monitorización constituye un elemento clave para detectar errores, analizar el comportamiento del sistema y anticipar posibles incidencias.

La plataforma de despliegue utilizada proporciona herramientas integradas de visualización de registros (*logs*), que permiten consultar en tiempo real la salida del servidor y detectar errores durante la ejecución de la aplicación. Estos registros incluyen tanto eventos asociados a peticiones *HTTP* como mensajes generados por el sistema de *logging* de *Django*, facilitando la identificación de fallos y el seguimiento del flujo de ejecución.

Además, la infraestructura *PaaS* empleada gestiona automáticamente el reinicio del servicio en caso de fallo inesperado, contribuyendo a mantener la disponibilidad del sistema sin intervención manual. Este comportamiento mejora la resiliencia de la aplicación frente a errores puntuales o problemas temporales de ejecución.

En cuanto al mantenimiento evolutivo, el modelo de despliegue basado en integración continua permite aplicar actualizaciones de forma sencilla mediante nuevas versiones del código. Cada modificación incorporada al repositorio desencadena un nuevo proceso de construcción y despliegue, lo que facilita la corrección de errores y la incorporación progresiva de mejoras.

Finalmente, el diseño modular del sistema y la separación entre entornos de desarrollo y producción favorecen la realización de tareas de mantenimiento correctivo y evolutivo sin comprometer la estabilidad del servicio en funcionamiento.

En conjunto, las estrategias de monitorización y mantenimiento adoptadas permiten garantizar un funcionamiento estable y seguro de la aplicación, alineado con buenas prácticas actuales en el desarrollo y operación de aplicaciones web en la nube.

Capítulo 6

Pruebas y validación

El proceso de pruebas y validación tiene como finalidad comprobar que la aplicación desarrollada funciona correctamente y cumple con los requisitos definidos durante las fases de análisis y diseño. En este capítulo se describen las pruebas realizadas para verificar las principales funcionalidades del sistema, así como la validación de la aplicación desde el punto de vista del usuario final.

Las pruebas se han llevado a cabo de forma progresiva a lo largo del desarrollo del proyecto, centrándose principalmente en la verificación funcional, la integración de los distintos componentes y la experiencia de uso tanto en dispositivos de escritorio como en dispositivos móviles. Este enfoque iterativo ha permitido detectar y corregir errores de manera temprana, garantizando un funcionamiento estable de la aplicación.

En los siguientes apartados se detallan el plan de pruebas adoptado, las pruebas funcionales y de integración realizadas, así como una evaluación de la usabilidad y del rendimiento del sistema.

6.1. Plan de pruebas

El plan de pruebas del proyecto se ha basado en un enfoque manual e incremental, adaptado al alcance y a las características de la aplicación desarrollada. A medida que se implementaban nuevas funcionalidades, estas se probaban desde la interfaz web para comprobar su correcto funcionamiento y su integración con el resto del sistema.

Las pruebas se han realizado simulando escenarios de uso reales, teniendo en cuenta los distintos tipos de usuarios definidos en el sistema (usuarios invitados, usuarios registrados y colaboradores). Este planteamiento ha permitido validar los flujos principales de la aplicación, incluyendo el registro, inicio de sesión y recuperación de contraseña, la creación y gestión de *packs*, la colaboración entre usuarios, el acceso público mediante enlaces y códigos QR[17], así como la creación y el uso de salas *online*.

Desde el punto de vista metodológico, el plan de pruebas se ha estructurado en varias categorías que se desarrollan en las siguientes secciones:

- Pruebas funcionales, centradas en verificar el correcto comportamiento de cada módulo.
- Pruebas de integración, orientadas a comprobar la interacción entre los distintos componentes del sistema y los servicios externos.
- Pruebas de usabilidad, destinadas a evaluar la experiencia de usuario y la adaptación a distintos dispositivos.
- Evaluación básica de rendimiento, enfocada en el comportamiento del sistema en situaciones de uso real.

Aunque no se ha desarrollado un conjunto formal de pruebas automatizadas, el proceso de validación manual ha resultado adecuado para identificar errores funcionales, problemas de permisos, incidencias en la integración con servicios externos y comportamientos no deseados. La detección temprana de estos problemas ha facilitado su corrección antes de avanzar a fases posteriores del desarrollo, contribuyendo a la estabilidad global del sistema.

6.2. Pruebas funcionales

Las pruebas funcionales se han centrado en verificar el correcto funcionamiento de las principales funcionalidades de la aplicación desde el punto de vista del usuario. Estas pruebas se han realizado de forma manual, interactuando con la interfaz web desde distintos perfiles de usuario y comprobando que cada acción produce el resultado esperado según los requisitos definidos.

El objetivo principal ha sido validar el comportamiento observable del sistema, asegurando que las operaciones disponibles para el usuario se ejecutan correctamente y respetan las restricciones de acceso establecidas.

Gestión de usuarios y autenticación

Se han realizado pruebas sobre el registro, inicio de sesión, recuperación de contraseñas y cierre de sesión de usuarios. En concreto, se ha verificado que:

- Los usuarios pueden registrarse correctamente introduciendo datos válidos.
- Las credenciales incorrectas generan mensajes de error adecuados sin revelar información sensible.
- Solo los usuarios autenticados acceden a las vistas privadas del sistema.
- Los usuarios no autenticados son redirigidos automáticamente al formulario de inicio de sesión cuando intentan acceder a recursos protegidos.

En relación con el sistema de recuperación de contraseña, se ha comprobado que:

- La solicitud de restablecimiento puede realizarse introduciendo una dirección de correo electrónico válida.
- El sistema no revela si la dirección introducida está registrada, evitando la enumeración de usuarios.

- Se envía correctamente el correo electrónico con el enlace de restablecimiento.
- El enlace generado permite establecer una nueva contraseña cuando el *token* es válido.
- Los *tokens* caducados o ya utilizados no permiten reutilizar el enlace.
- Tras completar el proceso, el usuario puede iniciar sesión con la nueva contraseña.

Asimismo, se ha comprobado que el cierre de sesión invalida correctamente la sesión activa y restringe el acceso posterior a las funcionalidades privadas.

Creación y gestión de *packs* y acciones

Se ha probado la creación, edición y eliminación de *packs*, verificando que:

- Los datos introducidos se almacenan correctamente en la base de datos.
- El *pack* creado queda automáticamente asociado al usuario autenticado.
- La eliminación de un *pack* elimina también las acciones asociadas, manteniendo la integridad referencial.

Del mismo modo, se han validado las operaciones de creación, modificación y eliminación de acciones dentro de un *pack*, confirmando que:

- Las acciones se asocian correctamente al *pack* correspondiente.
- Las validaciones de formulario impiden el almacenamiento de datos incompletos o inválidos.
- Los cambios realizados se reflejan de forma inmediata en la interfaz.

El sistema de mensajes informativos ha sido verificado, comprobando que el usuario recibe retroalimentación clara tras cada operación.

Sistema de colaboración y permisos

Las pruebas funcionales han incluido la validación del sistema de colaboración entre usuarios. Para ello, se han simulado distintos escenarios con propietarios y colaboradores, comprobando que:

- El propietario puede añadir colaboradores con roles diferenciados.
- Los colaboradores con rol *editor* pueden modificar el contenido del *pack*.
- Los colaboradores con rol *lector* pueden acceder al contenido, pero no editarlo.
- Las operaciones restringidas permanecen inaccesibles incluso si se intenta acceder manualmente a las URLs correspondientes.

Estas pruebas han confirmado que el sistema de autorización a nivel de recurso funciona correctamente y que los permisos se aplican de forma consistente.

Acceso público mediante enlaces y códigos QR

Se ha probado el acceso a *packs* mediante enlaces públicos basados en *token* y códigos QR, verificando que:

- Los usuarios invitados pueden acceder al contenido sin autenticación.
- El sistema recupera correctamente el *pack* asociado al *token*.
- El acceso público se limita a un uso de solo lectura.
- La regeneración del *token* invalida los enlaces previamente generados.

Asimismo, se ha validado la correcta generación y descarga del código QR, comprobando que la URL codificada dirige correctamente a la vista pública del *pack*.

Salas *online*

Por último, se han realizado pruebas funcionales sobre la creación y uso de salas *online*. Se ha comprobado que:

- Los usuarios con permisos adecuados pueden crear una sala asociada a un *pack*.
- Los participantes pueden unirse correctamente mediante el enlace generado.
- El estado de la sala se sincroniza entre los usuarios conectados.
- Los eventos de juego (inicio, selección de pregunta o reto) se transmiten correctamente mediante comunicación en tiempo real.
- La conexión con el servicio de videollamada se establece de forma segura.

Estas pruebas han permitido confirmar que la funcionalidad de salas *online* se integra correctamente con el resto del sistema y mantiene coherencia tanto en el plano funcional como en el de la experiencia de usuario.

Resumen de resultados

En la Tabla 6.1 se presenta un resumen de las principales funcionalidades evaluadas y el resultado obtenido.

Funcionalidad	**Resultado**
Registro e inicio de sesión	Correcto
Recuperación de contraseña	Correcto
Gestión de *packs*	Correcto
Sistema de colaboración	Correcto
Acceso público por *token*	Correcto
Salas *online*	Correcto

Tabla 6.1: Resumen de pruebas funcionales realizadas

Estas pruebas han permitido confirmar que la aplicación cumple con los requisitos establecidos y que las funcionalidades principales operan de manera consistente y estable.

6.3. Pruebas de integración

Las pruebas de integración se han orientado a verificar el correcto funcionamiento conjunto de los distintos subsistemas que componen la aplicación. A diferencia de las pruebas funcionales, centradas en validar comportamientos aislados desde el punto de vista del usuario, estas pruebas evalúan la interacción entre módulos internos y servicios externos, asegurando la coherencia global del sistema.

Integración aplicación–base de datos

Se ha validado la correcta interacción entre la capa de aplicación y el sistema gestor de base de datos. En concreto, se han comprobado los siguientes aspectos:

- Persistencia correcta de entidades complejas con relaciones (usuarios, *packs*, acciones, colaboraciones y salas).
- Aplicación adecuada de restricciones de integridad referencial, como eliminaciones en cascada.
- Consistencia del estado de las salas *online* en base de datos durante cambios de estado.
- Ejecución correcta de migraciones en el entorno de producción.

Estas pruebas han permitido confirmar que el modelo de datos definido se integra correctamente con la lógica de aplicación y que no se producen inconsistencias entre el estado persistente y el comportamiento observable.

Integración autenticación–autorización–vistas

Se ha comprobado la correcta integración entre:

- El sistema de autenticación de *Django*.
- Las funciones de autorización a nivel de recurso.
- Las vistas responsables de modificar contenido.

En particular, se han validado escenarios en los que distintos usuarios intentan acceder a recursos compartidos, comprobando que:

- La autenticación se mantiene correctamente entre peticiones mediante el sistema de sesiones.
- Las comprobaciones de permisos se aplican antes de ejecutar operaciones críticas.
- No es posible eludir restricciones manipulando manualmente las URLs.

Estas pruebas garantizan que el flujo completo autenticación $\rightarrow$ autorización $\rightarrow$ ejecución de lógica se comporta de forma coherente.

Integración *WebSocket*–estado de la sala

En el caso de las salas *online*, se ha validado la integración entre:

- El modelo `VideoRoom` en base de datos.
- Los consumidores de *Django Channels*.
- La interfaz cliente mediante *WebSocket*.

Se han realizado pruebas en las que varios participantes se conectan simultáneamente a una sala, verificando que:

- El estado inicial se sincroniza correctamente al unirse un nuevo participante.
- Los cambios de estado (inicio de partida, selección de pregunta) se propagan a todos los clientes conectados.
- El estado persistente de la sala permanece coherente con el estado mostrado en la interfaz.

Estas pruebas han permitido asegurar que la comunicación en tiempo real no rompe la consistencia del sistema.

Integración con el servicio externo de videollamada

Se ha validado la integración entre la aplicación y el servicio externo de videollamada basado en *Jitsi (8x8 JaaS)*. En particular, se ha comprobado que:

- El *token JWT* generado en el servidor permite el acceso correcto a la sala.
- El identificador de sala se corresponde con el modelo interno `VideoRoom`.
- Los usuarios con rol de anfitrión reciben permisos adecuados en la videollamada.
- La conexión HTTPS y el uso de *WebSocket* seguro no generan conflictos de seguridad en el navegador.

Estas pruebas han sido fundamentales para garantizar que la integración con un servicio externo funcione correctamente y que la experiencia de uso sea fluida.

Integración con el servicio externo de correo

Se ha confirmado la integración entre la aplicación y el servicio externo de envío de correo electrónico utilizado para el sistema de recuperación de contraseñas. En concreto, se han comprobado los siguientes aspectos:

- Correcta configuración del `EMAIL_BACKEND` en el entorno de producción.
- Disponibilidad de las variables de entorno necesarias, como `SENDGRID_API_KEY` y `DEFAULT_FROM_EMAIL`.
- Envío efectivo del correo de restablecimiento de contraseña tras la solicitud del usuario.
- Generación correcta del enlace de recuperación incluido en el mensaje.
- Registro adecuado de errores en caso de incidencia en la comunicación con la *API* externa.

Estas pruebas han permitido confirmar que la integración con el servicio de correo funciona de forma coherente con el sistema de autenticación, garantizando la entrega de mensajes transaccionales sin comprometer la seguridad ni la arquitectura interna de la aplicación.

Integración en entorno de producción

Finalmente, se han realizado pruebas de integración en el entorno desplegado en la plataforma *Render*, verificando:

- Correcta ejecución de migraciones al iniciar el servicio.
- Disponibilidad de archivos estáticos tras la ejecución de `collectstatic`.
- Conectividad con la base de datos en producción.
- Funcionamiento correcto de conexiones HTTPS y WSS.

Estas pruebas han permitido validar el comportamiento completo del sistema en condiciones reales de despliegue.

Conclusión

Las pruebas de integración confirman que los distintos componentes del sistema —modelo de datos, lógica de aplicación, comunicación en tiempo real y servicios externos— funcionan de forma coordinada y coherente. No se han detectado inconsistencias significativas entre módulos, lo que respalda la solidez de la arquitectura adoptada.

6.4. Pruebas de usabilidad

Las pruebas de usabilidad se han orientado a evaluar la experiencia de uso de la aplicación desde la perspectiva del usuario final, verificando que la interacción con el sistema resulte clara, intuitiva y eficiente. Estas pruebas se han realizado de manera práctica mediante el uso continuado de la aplicación en distintos dispositivos y escenarios de uso, analizando especialmente la facilidad de navegación y la comprensión de las funcionalidades disponibles.

Criterios de evaluación

La evaluación se ha basado en principios generales de usabilidad ampliamente aceptados en el ámbito del diseño de interfaces, tales como:

- **Simplicidad**: reducción del número de pasos necesarios para completar acciones clave.
- **Consistencia**: mantenimiento de una estructura visual y de navegación homogénea en todas las vistas.
- **Visibilidad del estado del sistema**: información clara sobre el resultado de las acciones realizadas.
- **Prevención de errores**: validación de formularios y restricciones de acceso antes de ejecutar operaciones críticas.

Estos criterios han servido como marco de referencia para analizar si las decisiones de diseño adoptadas en fases anteriores se traducen en una experiencia de uso efectiva.

Evaluación en dispositivos de escritorio

En dispositivos de escritorio, se ha comprobado que la navegación entre las distintas secciones resulta clara e intuitiva. Las principales tareas evaluadas han sido:

- Registro e inicio de sesión.
- Creación y edición de *packs*.
- Gestión de colaboradores.
- Creación y uso de salas *online*.

Se ha verificado que estas acciones pueden realizarse sin ambigüedad en los pasos necesarios y que el sistema proporciona retroalimentación inmediata mediante mensajes informativos tras cada operación.

Evaluación en dispositivos móviles

La evaluación en dispositivos móviles ha resultado especialmente relevante, dado que el acceso a los *packs* mediante enlaces públicos o códigos QR se realiza mayoritariamente desde este tipo de dispositivos.

Se ha comprobado que:

- La interfaz se adapta correctamente a pantallas de menor tamaño.
- Los elementos interactivos mantienen un tamaño adecuado para su uso táctil.
- El contenido del juego se presenta de forma legible sin necesidad de desplazamientos excesivos.

El diseño adaptable implementado garantiza que la experiencia de juego sea fluida tanto en orientación vertical como horizontal, manteniendo coherencia visual y funcional.

Resultados

Las pruebas realizadas indican que la aplicación ofrece una experiencia de uso coherente y consistente en distintos dispositivos. Las tareas principales pueden completarse sin dificultad significativa y la diferenciación entre vistas públicas y privadas facilita la comprensión de los permisos disponibles.

Aunque no se ha llevado a cabo un estudio formal con métricas cuantitativas o pruebas controladas con un grupo amplio de usuarios, la evaluación práctica realizada durante el desarrollo confirma que la interfaz cumple con los criterios básicos de usabilidad definidos para el proyecto.

6.5. Evaluación de rendimiento

La evaluación del rendimiento del sistema se ha realizado con el objetivo de analizar la capacidad de la aplicación para responder de forma rápida y estable bajo condiciones normales de uso. Dado el alcance académico del proyecto, la evaluación se ha centrado en escenarios reales

de utilización, combinando observación cualitativa con mediciones básicas de tiempos de carga en el entorno de producción.

Metodología de medición

Las mediciones se han realizado utilizando las herramientas de desarrollo del navegador (pestaña *Network*), con la caché deshabilitada y sin limitación artificial de red (*No throttling*).

En el caso de navegación entre vistas, se ha tomado como referencia el evento *Load*. Para aquellas operaciones que implican el envío de formularios, como la creación de un *pack*, se ha considerado el tiempo total de la petición HTTP *POST*, ya que representa el tiempo real de procesamiento en el servidor. En todos los casos se han realizado varias mediciones desde diferentes dispositivos, calculando posteriormente un tiempo medio aproximado.

Este procedimiento permite obtener una estimación realista del comportamiento del sistema en condiciones normales de acceso desde un navegador estándar.

Es importante señalar que los tiempos observados dependen en parte de las características del equipo cliente y de la conexión de red utilizada durante las pruebas. Las mediciones principales se realizaron desde un ordenador portátil con procesador *Intel Core i7* de 8ª generación, 8 GB de memoria *RAM* y sistema operativo *Windows 11 Pro*, utilizando el navegador *Google Chrome* en su versión estable.

La conexión empleada corresponde a una red doméstica de fibra óptica, con conexión *WiFi* 5 y un *throughput TCP* efectivo aproximado de 244 *Mbps* en descarga y 212 *Mbps* en subida, medido mediante herramientas estándar de test de velocidad. No se aplicó limitación artificial de red durante las pruebas.

Por tanto, los valores obtenidos deben interpretarse como una referencia representativa bajo condiciones normales de acceso desde un entorno doméstico, pudiendo variar en función del dispositivo, la latencia de red y la carga del servidor en el momento de la medición.

Rendimiento en operaciones habituales

Se ha analizado el comportamiento del sistema en las operaciones más frecuentes, tales como:

- Navegación entre vistas.
- Creación y edición de *packs*.
- Gestión de acciones.
- Acceso público mediante *token*.
- Creación y unión a salas *online*.

Los tiempos medios observados en entorno de producción se resumen en la Tabla 6.2.

Acción	Tiempo medio observado
Página principal	1.11 s
Página de detalles de *pack*	1.61 s
Crear *pack* (Formulario *POST*)	1.06 s
Acceso público mediante *token*	1.30 s
Creación de sala *online*	4.76 s

Tabla 6.2: Tiempos medios observados en entorno de producción.

Como puede observarse, las operaciones que implican consultas simples a la base de datos presentan tiempos de respuesta adecuados para una aplicación web desplegada en un entorno *PaaS*. Los valores registrados se sitúan dentro de rangos habituales para aplicaciones alojadas en infraestructuras compartidas.

En el caso de la creación de salas *online*, el tiempo es superior debido a varios factores: la generación del *token* de autenticación para el servicio externo de videollamada, la serialización de las acciones activas del *pack*, la inicialización de la conexión *WebSocket* y la carga inicial del entorno de comunicación en tiempo real. A pesar de ello, el tiempo registrado se mantiene dentro de valores aceptables para una aplicación web interactiva.

Durante las pruebas no se han observado bloqueos, errores derivados de sobrecarga ni degradaciones significativas en el uso habitual.

Comunicación en tiempo real

En el caso de las salas *online*, se ha evaluado el comportamiento del sistema con múltiples participantes conectados simultáneamente. Se ha comprobado que:

- La sincronización del estado mediante *WebSocket* se realiza de forma inmediata.
- Los eventos de juego se propagan correctamente a todos los clientes conectados.
- La carga asociada a la comunicación en tiempo real no afecta al rendimiento general de la aplicación.

Es importante destacar que la transmisión de audio y vídeo se gestiona a través del servicio externo integrado, lo que descarga al servidor principal de la aplicación de dicha responsabilidad y contribuye a mantener un consumo de recursos moderado.

Entorno de producción

En el entorno desplegado en la plataforma *Render*, la aplicación ha mostrado un comportamiento estable bajo uso continuado. La gestión automática de procesos y el reinicio del servicio en caso de fallo contribuyen a mantener la disponibilidad del sistema.

Asimismo, la separación entre el servidor de aplicación, la base de datos y el servicio externo de videollamada favorece una distribución adecuada de la carga, reduciendo el riesgo de cuellos

de botella.

Limitaciones y posibles mejoras

No se han realizado pruebas de carga intensiva ni simulaciones de un gran número de usuarios concurrentes. Este tipo de análisis requeriría herramientas específicas y un entorno de pruebas controlado, que excede el alcance del presente proyecto.

Como línea de mejora futura, podría plantearse la incorporación de:

- Pruebas automatizadas de carga.
- Monitorización continua de tiempos de respuesta.
- Evaluación del comportamiento bajo alta concurrencia en salas *online*.

En conjunto, la evaluación realizada permite concluir que el rendimiento del sistema es adecuado para los escenarios de uso previstos y coherente con los objetivos y dimensión del proyecto.

Capítulo 7

Resultados y discusión

En este capítulo se analizan los resultados obtenidos tras el desarrollo del proyecto, evaluando el grado de cumplimiento de los objetivos planteados inicialmente y reflexionando sobre las limitaciones encontradas y las lecciones aprendidas durante el proceso. Este análisis permite valorar de forma crítica el trabajo realizado y contextualizar los resultados alcanzados dentro del alcance académico del proyecto y los recursos disponibles.

Además de comprobar el correcto funcionamiento del sistema, se examinan las decisiones técnicas adoptadas, la integración de las distintas tecnologías empleadas y la viabilidad global de la solución desarrollada. De este modo, se ofrece una visión completa del proyecto, considerando tanto sus fortalezas como sus posibles áreas de mejora.

7.1. Cumplimiento de objetivos

El principal objetivo del proyecto consistía en desarrollar una plataforma web interactiva que permitiera jugar a *Verdad o Reto* de forma accesible, personalizable y adaptable a distintos dispositivos, superando las limitaciones de versiones existentes. Este objetivo se ha alcanzado mediante la implementación de una aplicación web funcional que permite el acceso inmediato al juego tanto en contextos presenciales como remotos, ofreciendo una experiencia coherente en dispositivos de escritorio y móviles.

En relación con los objetivos específicos definidos inicialmente, puede afirmarse lo siguiente:

- **Diseño de una interfaz clara y adaptable:** se ha desarrollado una interfaz web *responsive* que se adapta a distintos tamaños de pantalla, garantizando una experiencia de uso adecuada tanto en ordenadores como en dispositivos móviles.
- **Creación y gestión de *packs*:** la aplicación permite a los usuarios registrados crear, editar y eliminar *packs* personalizados de preguntas y retos, así como gestionar su contenido de forma estructurada.
- **Acceso público mediante enlaces y códigos QR:** se ha implementado un sistema basado en *tokens* únicos que permite compartir *packs* sin exponer identificadores internos, facilitando

el acceso mediante enlaces directos y códigos QR.

- **Interacción en tiempo real mediante salas *online*:** se han incorporado funcionalidades de comunicación en tiempo real, combinando *Django Channels* para la sincronización del estado del juego y un servicio externo de videollamada para la interacción audiovisual.
- **Garantía de seguridad en la gestión de usuarios y datos:** se han aplicado mecanismos de autenticación, autorización y protección frente a ataques comunes, además de una correcta gestión de claves secretas y variables de entorno en producción.
- **Despliegue en entorno accesible desde Internet:** la aplicación ha sido desplegada en una plataforma *PaaS*, permitiendo su acceso público mediante dominio y conexión segura HTTPS.

A la luz de los resultados obtenidos, puede afirmarse que los objetivos funcionales y técnicos establecidos al inicio del proyecto se han alcanzado de manera satisfactoria, tanto desde el punto de vista del desarrollo como de su despliegue y utilización en un entorno real.

7.2. Limitaciones

A pesar de los resultados obtenidos y del correcto funcionamiento del sistema en los escenarios previstos, el proyecto presenta una serie de limitaciones que deben ser consideradas.

En primer lugar, el proceso de pruebas se ha centrado principalmente en validaciones funcionales y de usabilidad realizadas de forma manual, sin la implementación de un conjunto formal de pruebas automatizadas. Aunque este enfoque ha sido suficiente para garantizar la estabilidad del sistema en el contexto del proyecto, limita la capacidad de detección sistemática de errores ante modificaciones futuras del código y dificulta la validación continua en entornos de mayor complejidad.

En relación con la funcionalidad de salas *online*, la integración se apoya en un servicio externo de videollamada. Esta decisión ha permitido simplificar la arquitectura y evitar la gestión de infraestructura propia para transmisión audiovisual, pero introduce una dependencia de terceros en términos de disponibilidad, configuración y posibles cambios en las condiciones del servicio. Asimismo, la escalabilidad del sistema en este ámbito queda condicionada por las limitaciones del proveedor externo.

Otra limitación relevante se encuentra en el análisis de rendimiento. La evaluación realizada ha sido principalmente cualitativa, complementada con mediciones básicas de tiempos de carga, sin llevar a cabo pruebas de carga intensiva ni simulaciones de alta concurrencia. En consecuencia, no se dispone de datos concluyentes sobre el comportamiento del sistema ante un número elevado de usuarios simultáneos.

Desde el punto de vista funcional, el sistema de gestión de permisos y colaboración, aunque operativo y seguro, presenta un modelo relativamente simple basado en roles predefinidos. En escenarios más complejos podría resultar necesario incorporar un sistema de permisos más granular, con control detallado sobre acciones específicas y configuración avanzada de accesos.

Por último, el proyecto se ha desarrollado con un alcance claramente delimitado en el contexto académico, lo que implica que aspectos como la internacionalización, la personalización avanzada de la interfaz o la integración con servicios adicionales no han sido abordados en profundidad.

7.3. Lecciones aprendidas

El desarrollo del proyecto ha permitido adquirir una visión completa del ciclo de vida de una aplicación web, abarcando desde el análisis inicial y el diseño del modelo de datos hasta la implementación, integración de servicios externos y despliegue en producción. Esta experiencia ha puesto de manifiesto la importancia de planificar la arquitectura del sistema desde fases tempranas, definiendo correctamente las responsabilidades de cada componente y anticipando posibles necesidades de escalabilidad.

Una de las lecciones más relevantes ha sido la necesidad de tomar decisiones técnicas fundamentadas y, al mismo tiempo, mantener flexibilidad para adaptarse a los problemas reales que surgen durante el desarrollo. La evolución en la integración de las salas *online* y la adopción de una solución más adecuada para la comunicación en tiempo real ilustran cómo la reevaluación de decisiones iniciales puede conducir a una arquitectura más estable y sostenible.

Asimismo, el proyecto ha evidenciado la relevancia de separar adecuadamente la lógica de negocio, la presentación y la gestión de la comunicación en tiempo real. La combinación de *Django*, *Django Channels* y servicios externos ha requerido comprender cómo interactúan distintos paradigmas de programación (sincrónico y asíncrono), reforzando la importancia de diseñar sistemas modulares y desacoplados.

Desde la perspectiva de la experiencia de usuario, se ha confirmado que aspectos como la simplicidad del acceso, el diseño adaptable y la claridad en la interacción influyen directamente en la percepción global del sistema. La incorporación de enlaces públicos y códigos QR ha demostrado cómo pequeñas decisiones de diseño pueden tener un impacto significativo en la usabilidad.

Finalmente, el proyecto ha contribuido a consolidar conocimientos en áreas clave como seguridad, gestión de entornos de producción y despliegue en la nube. La experiencia adquirida en la configuración de variables de entorno, manejo de claves secretas, integración continua básica y resolución de incidencias en producción ha reforzado la autonomía técnica y la capacidad de análisis ante problemas complejos.

Capítulo 8

Conclusiones y trabajos futuros

Este capítulo recoge las conclusiones finales derivadas del desarrollo del proyecto, sintetizando las principales aportaciones técnicas y funcionales del sistema implementado. Asimismo, se plantean posibles líneas de trabajo futuro que permitirían ampliar o mejorar la solución desarrollada, atendiendo a las limitaciones identificadas y a nuevas oportunidades de evolución.

Como cierre académico del Trabajo Fin de Grado, se reflexiona también sobre las competencias técnicas y profesionales adquiridas durante el proceso, destacando la experiencia obtenida en el diseño, implementación y despliegue de aplicaciones web en un entorno real.

8.1. Conclusiones

El desarrollo de este Trabajo Fin de Grado ha permitido diseñar e implementar una versión web interactiva basada en el juego social *Verdad o Reto*, ampliando su alcance tradicional mediante la incorporación de funcionalidades digitales avanzadas. La aplicación resultante no solo reproduce la dinámica del juego en formato web, sino que introduce mecanismos de personalización, colaboración y comunicación remota que enriquecen significativamente la experiencia de uso.

Desde el punto de vista técnico, el proyecto ha integrado de forma coherente distintas tecnologías del ecosistema web moderno, combinando desarrollo *backend* con *Django*, gestión de comunicación en tiempo real mediante *Django Channels* e integración de servicios externos para videollamada. Asimismo, se ha abordado el despliegue en un entorno de producción real, considerando aspectos de seguridad, gestión de configuraciones, almacenamiento persistente y acceso seguro mediante *HTTPS*.

La solución obtenida ha demostrado ser funcional y estable en los escenarios de uso previstos, validando la arquitectura adoptada y las decisiones técnicas tomadas durante el desarrollo. Además, el proyecto pone de manifiesto la viabilidad de aplicar tecnologías web actuales para la construcción de aplicaciones sociales interactivas accesibles desde distintos dispositivos y contextos de uso.

En definitiva, el trabajo realizado no solo cumple los objetivos planteados inicialmente, sino que consolida una base técnica sólida sobre la que podrían desarrollarse futuras ampliaciones o adaptaciones del sistema.

8.2. Líneas de trabajo futuro

A partir de los resultados obtenidos y de las limitaciones identificadas, se plantean diversas líneas de evolución que permitirían ampliar y mejorar la aplicación desarrollada.

Evolución funcional del sistema

Una posible ampliación consiste en la incorporación de un sistema de estadísticas que permita registrar información relevante sobre el uso de la aplicación, como el número de partidas jugadas, el tiempo promedio de duración de las salas o el número de accesos mediante códigos QR. Estos datos podrían utilizarse tanto con fines informativos para los usuarios como para el análisis del comportamiento del sistema.

Asimismo, podría implementarse un sistema de turnos automático en las salas *online*, incluyendo control del orden de participación y temporizadores para responder a cada pregunta o reto. Esta mejora permitiría estructurar mejor la dinámica del juego y reducir la dependencia de la gestión manual por parte del anfitrión.

Otra línea de evolución interesante sería la creación de una galería pública de *packs*, con filtros por categoría, sistema de valoraciones y posibilidad de explorar contenido generado por otros usuarios. Esta funcionalidad convertiría la plataforma en un entorno más colaborativo y orientado a la comunidad.

Desde el punto de vista de la experiencia de usuario, podrían incorporarse elementos audiovisuales adicionales, como música de fondo, efectos sonoros o animaciones, con el objetivo de enriquecer la interacción y hacer la experiencia más atractiva.

Finalmente, la integración con redes sociales permitiría compartir partidas, resultados o *packs* de forma directa, ampliando la visibilidad de la aplicación y fomentando su difusión.

Mejoras técnicas y de calidad

En el ámbito técnico, una mejora relevante sería la incorporación de pruebas automatizadas, incluyendo pruebas unitarias y de integración. Esto permitiría aumentar la fiabilidad del sistema y facilitar su mantenimiento ante futuras modificaciones.

También podría optimizarse la lógica de selección de preguntas para evitar repeticiones dentro de una misma partida, incorporando mecanismos de control de historial o exclusión temporal de elementos ya utilizados.

Escalabilidad y robustez

En términos de seguridad y control de acceso, podrían implementarse salas protegidas mediante contraseña, ampliando las posibilidades de configuración y privacidad de las partidas.

Asimismo, el mecanismo de expiración automática de salas podría reforzarse mediante tareas programadas o servicios específicos de limpieza periódica, mejorando la gestión de recursos y garantizando un control más preciso sobre la duración de las sesiones activas.

Competencias

La realización de este Trabajo Fin de Grado ha permitido consolidar competencias técnicas y profesionales fundamentales en el ámbito de la Ingeniería Telemática. Desde el punto de vista técnico, se han aplicado conocimientos relacionados con el diseño de arquitecturas web basadas en el patrón *MVT*, la gestión de bases de datos relacionales, la implementación de mecanismos de autenticación y autorización, y la integración de comunicación en tiempo real mediante tecnologías asíncronas.

Asimismo, el proyecto ha requerido la integración de servicios externos y la configuración de un entorno de producción real, abordando aspectos como la gestión de variables de entorno, protección de claves secretas, despliegue en plataformas *PaaS* y resolución de incidencias en entornos *cloud*.

En el ámbito profesional, se han desarrollado competencias relacionadas con la planificación progresiva del trabajo, la toma de decisiones técnicas fundamentadas, la adaptación ante problemas imprevistos y la evaluación crítica de soluciones alternativas. La elaboración de la presente memoria ha reforzado además las habilidades de comunicación técnica, estructuración de documentación y presentación formal de un proyecto de ingeniería.

Estas competencias reflejan la capacidad para abordar de forma autónoma el desarrollo completo de una aplicación *software*, desde su concepción inicial hasta su puesta en producción.

Bibliografía

[1] P. S. Foundation. «Python Official Documentation. »dirección: https://www.python.org/.

[2] D. S. Foundation. «Django Documentation. »dirección: https://docs.djangoproject.com/.

[3] O. Foundation. «Cross-site scripting (XSS). »dirección: https://owasp.org/www-community/attacks/xss.

[4] O. Foundation. «SQL Injection. »dirección: https://owasp.org/www-community/attacks/SQL_Injection.

[5] Django Software Foundation, *Daphne - HTTP, HTTP2 and WebSocket protocol server for ASGI*, https://github.com/django/daphne, Consultado en marzo de 2026, 2024.

[6] D. S. Foundation, *Django Architecture: The Model–View–Template Pattern*, 2025. dirección: https://docs.djangoproject.com/en/stable/faq/general/#what-happened-to-the-view-controller.

[7] T. Reenskaug, *Models - Views - Controllers*, Xerox PARC technical note, Original MVC pattern description, 1979.

[8] M. D. Network. «Guía de HTML5. »dirección: https://developer.mozilla.org/en-US/docs/Glossary/HTML5.

[9] W3C, *CSS3 Specification*, 2025. dirección: https://www.w3.org/TR/css/.

[10] E. Marcotte, «Responsive Web Design,» *A List Apart*, 2010. dirección: https://alistapart.com/article/responsive-web-design/.

[11] W. W. W. C. (W3C). «Responsive Web Design Basics. »dirección: https://www.w3.org/.

[12] M. D. Network, *JavaScript Documentation (MDN)*, 2025. dirección: https://developer.mozilla.org/en-US/docs/Web/JavaScript.

[13] W. W. W. C. (W3C), «Document Object Model (DOM) Living Standard,» W3C, inf. téc., 2024. dirección: https://dom.spec.whatwg.org/.

[14] O. Foundation. «Cross-Site Request Forgery (CSRF). »dirección: https://owasp.org/www-community/attacks/csrf.

[15] S. Consortium, *SQLite Documentation*, 2025. dirección: https://www.sqlite.org/docs.html.

[16] P. G. D. Group, *PostgreSQL Documentation*, 2025. dirección: https://www.postgresql.org/docs/.

[17] International Organization for Standardization, «ISO/IEC 18004:2015 - Information technology – Automatic identification and data capture techniques – QR Code bar code symbology specification,» ISO, inf. téc., 2015.

[18] J. Community. «Jitsi Meet Documentation. »dirección: https://jitsi.github.io/handbook/docs/intro/.

[19] N. I. of Standards y Technology. «The NIST Definition of Cloud Computing. »dirección: https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-145.pdf.

[20] Render, Inc. «Render Documentation.» Accedido: 13-02-2026. dirección: https://render.com/docs.

[21] Twilio Inc. «SendGrid Web API Documentation.» Último acceso: marzo 2026. dirección: https://docs.sendgrid.com/api-reference.

[22] Twilio Inc. «SendGrid: Email API & Email Delivery Service.» Último acceso: marzo 2026. dirección: https://sendgrid.com/.

[23] I. 8x8. «8x8 Jitsi as a Service (JaaS) Developer Documentation. »dirección: https://developer.8x8.com/jaas/.

[24] P. S. Foundation. «venv — Creation of virtual environments. »dirección: https://docs.python.org/3/library/venv.html.

[25] S. Chacon y B. Straub, *Pro Git*. Apress, 2023. dirección: https://git-scm.com/book/en/v2.

[26] GitHub, Inc. «GitHub Documentation. »dirección: https://docs.github.com/.

[27] GitHub, Inc. «GitHub. »dirección: https://github.com/.

Apéndice A

Manual de usuario

A.1. Introducción

El presente apéndice tiene como finalidad proporcionar una guía práctica de uso de la aplicación web desarrollada en el marco de este Trabajo Fin de Grado. Se describen de forma detallada las funcionalidades disponibles para el usuario final, así como el flujo de interacción habitual dentro del sistema.

La aplicación permite jugar a *Verdad o Reto* tanto en modo local como en modalidad remota mediante salas *online*, pudiendo acceder a ella desde distintos dispositivos. Además, ofrece la posibilidad de crear y gestionar *packs* personalizados de preguntas y retos. El sistema está diseñado para ser accesible desde cualquier navegador web moderno, sin necesidad de instalar *software* adicional.

El manual está orientado a usuarios finales que deseen utilizar la plataforma, independientemente de su nivel técnico. Por ello, las distintas secciones describen paso a paso las acciones necesarias para acceder a la aplicación, registrarse e iniciar sesión, crear y manejar contenido, compartir enlaces públicos y participar en partidas *online*.

Con el objetivo de facilitar la comprensión, se incluyen capturas de pantalla que ilustran visualmente cada proceso descrito. Estas imágenes corresponden a la versión desplegada en entorno de producción y reflejan el comportamiento real del sistema desde un dispositivo de escritorio.

A.2. Acceso a la aplicación

El acceso a la aplicación se realiza desde cualquier navegador web moderno a través de la URL `https://verdadreto.onrender.com/`. No es necesaria ninguna configuración ni instalación adicional, ya que se trata de una aplicación web accesible mediante el protocolo seguro *HTTPS*.

Al introducir la dirección web en la barra del navegador, el usuario accede a la página

principal del sistema, desde la cual puede navegar al resto de las funcionalidades disponibles. La interfaz presenta un diseño claro e intuitivo, adaptado tanto a dispositivos de escritorio como a dispositivos móviles.

La Figura A.1 muestra la página principal de la aplicación tras acceder a la dirección pública.

Figura A.1: Página principal de la aplicación.

Esta vista presenta la información introductoria del sistema y resume sus principales características. Desde el menú de navegación superior es posible acceder a los formularios de registro e inicio de sesión, en caso de que el usuario disponga de una cuenta.

También es posible acceder a los *packs* personales del usuario a través del botón principal **"Jugar ahora"**, redirigiendo automáticamente al formulario de inicio de sesión en caso de que no exista una sesión activa.

A.3. Registro e inicio de sesión

Para acceder a funcionalidades avanzadas, como la creación y gestión de contenido, manejo de QRs o la creación de salas *online*, es necesario disponer de una cuenta autenticada.

El proceso de registro se inicia al seleccionar la opción **"Registrarse"** disponible en el menú de navegación. A continuación, el sistema muestra el formulario representado en la Figura A.2.

A.3.1. Registro de usuario

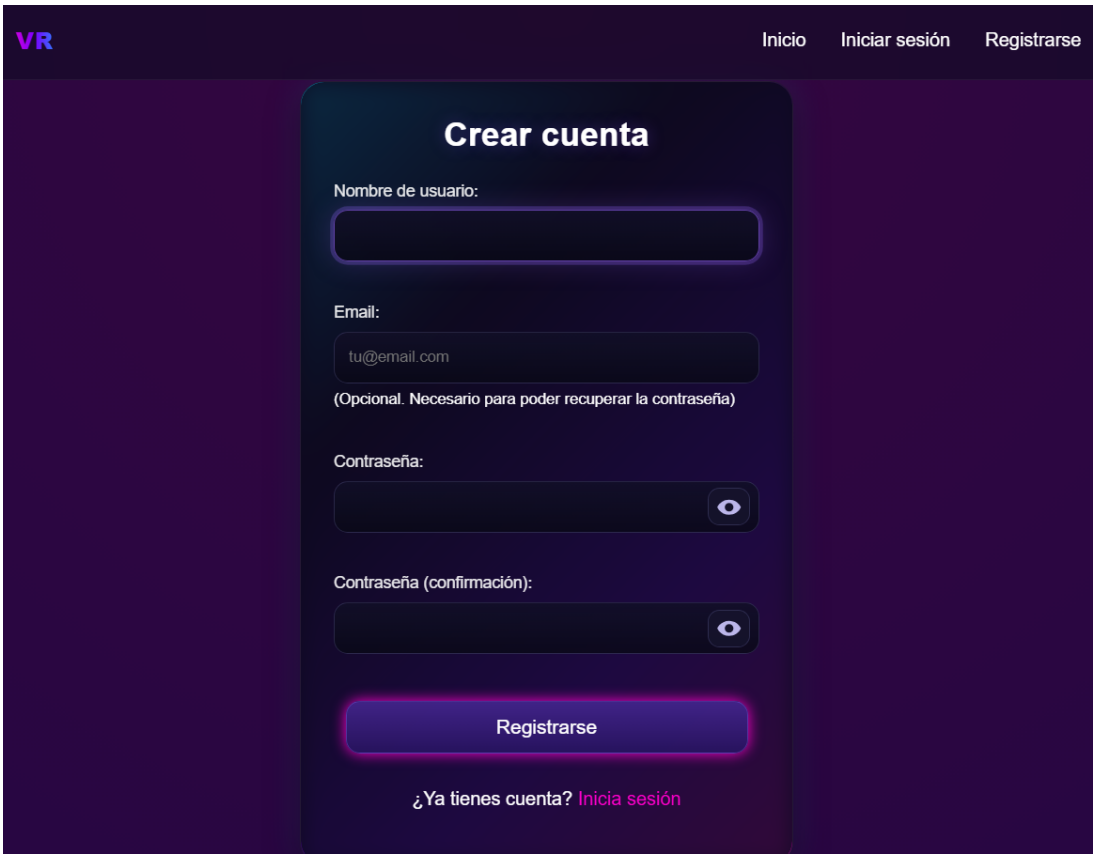

Figura A.2: Formulario de registro.

En este formulario, el usuario debe introducir un nombre de usuario y una contraseña, que deberá confirmar en un segundo campo. De manera opcional, puede añadir una dirección de correo electrónico, que será necesaria si se desea utilizar la funcionalidad de recuperación de contraseña.

Una vez completados los campos correspondientes, si no se detecta ningún error en la validación de los datos, el usuario queda registrado en la base de datos y es redirigido al formulario de inicio de sesión.

En caso de que el nombre de usuario ya esté registrado, o haya algún error en la confirmación de credenciales, el sistema informará al usuario con los mensajes de error correspondientes.

A.3.2. Inicio de sesión

Los usuarios que ya dispongan de una cuenta pueden autenticarse mediante la opción **“Iniciar sesión”**. El formulario es el representado en la Figura A.3.

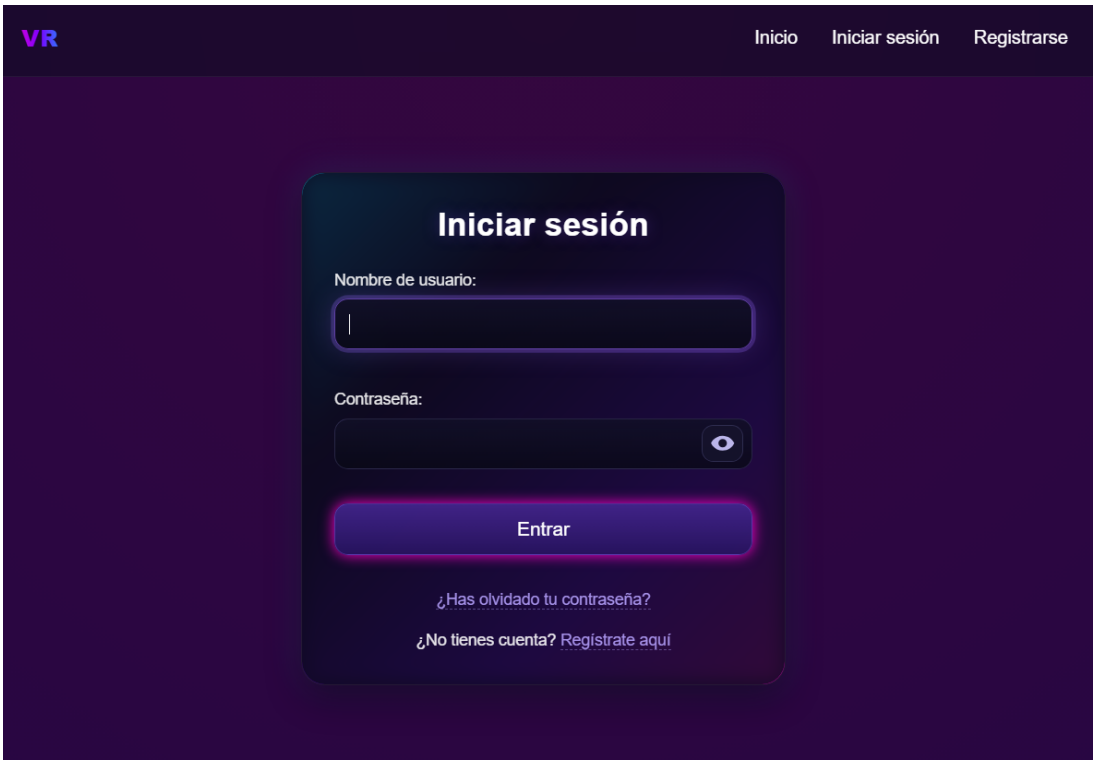

Figura A.3: Formulario de inicio de sesión.

Tras completar correctamente los campos, el sistema verifica las credenciales mediante el mecanismo de autenticación proporcionado por *Django*. Si las credenciales son válidas, se inicia una sesión autenticada que permite acceder a las funcionalidades privadas del sistema.

En caso de no recordar la contraseña, el usuario puede seleccionar la opción **"¿Has olvidado tu contraseña?"**, que inicia el proceso de recuperación descrito en la siguiente subsección.

De igual manera que en el formulario de registro, si se introducen credenciales incorrectas o se produce cualquier incidencia durante el proceso de autenticación, se informará con los mensajes necesarios al usuario.

A.3.3. Recuperación de contraseña

En caso de que el usuario no recuerde su contraseña, puede iniciar el proceso de recuperación mediante el formulario mostrado en la Figura A.4, indicando la dirección de correo electrónico asociada a su cuenta.

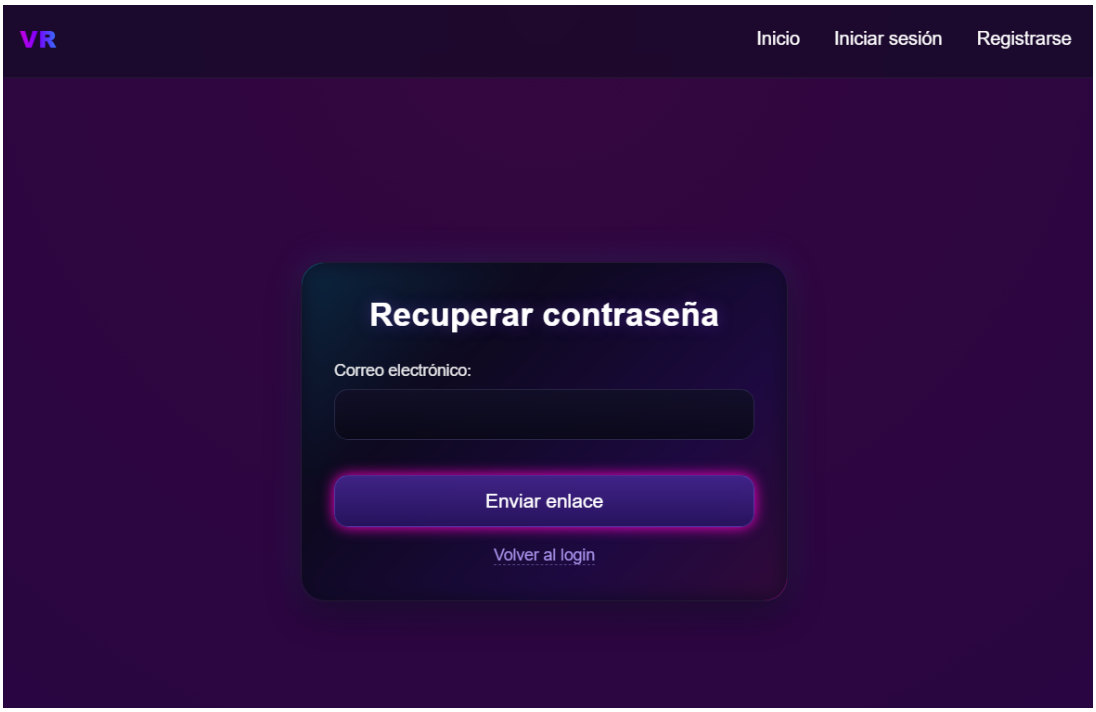

Figura A.4: Formulario de recuperación de contraseña.

Una vez enviada la solicitud, el sistema remitirá un mensaje a la dirección indicada con un enlace para restablecer la contraseña. Este enlace conduce a una página segura donde el usuario podrá introducir y confirmar una nueva contraseña.

Si el proceso se completa correctamente, la contraseña anterior queda invalidada y el usuario podrá iniciar sesión utilizando la nueva credencial.

A.4. Panel de usuario: Mis packs

Una vez autenticado, el usuario tiene acceso a la sección **Mis packs**, donde se muestran todos los *packs* creados en su cuenta. Esta vista actúa como panel principal de gestión del contenido personalizado del sistema.

La Figura A.5 muestra la interfaz correspondiente.

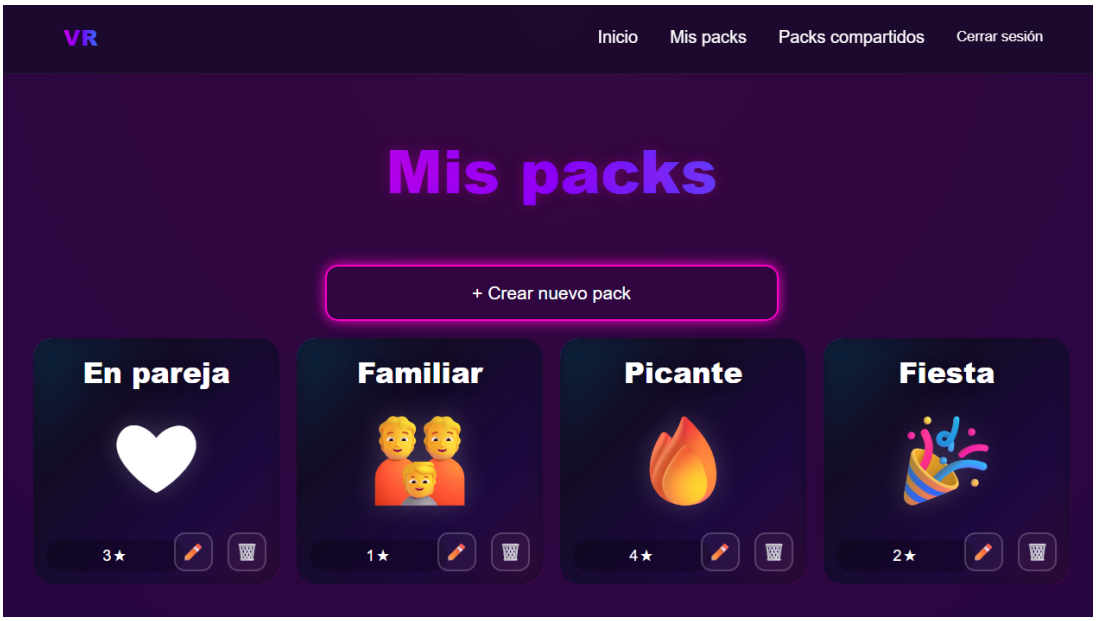

Figura A.5: Vista de gestión de packs personales del usuario.

En esta pantalla, cada *pack* se representa como una tarjeta individual que incluye su nombre, un elemento gráfico representativo y el nivel de dificultad asociado.

El usuario puede interactuar con cada *pack* de diferentes formas:

- **Acceso a la página pública del pack:** al seleccionar la tarjeta completa, el sistema redirige a la vista pública del *pack*, permitiendo iniciar el juego en modo local.
- **Edición:** el icono de lápiz permite acceder a la página de detalles del *pack*, donde es posible modificar su contenido y acceder a otras funcionalidades.
- **Eliminación:** el icono de papelera permite eliminar el *pack* del sistema. Esta acción requiere confirmación previa.

Asimismo, en la parte superior de la pantalla se encuentra el botón **"+ Crear nuevo pack"**, que permite generar un nuevo *pack* personalizado. Al seleccionarlo, el sistema redirige al formulario correspondiente para la creación del nuevo *pack*.

A.4.1. Creación de un nuevo pack

La creación de un nuevo *pack* se inicia seleccionando el botón **"+ Crear nuevo pack"**, que redirige al formulario correspondiente, mostrado en la Figura A.6.

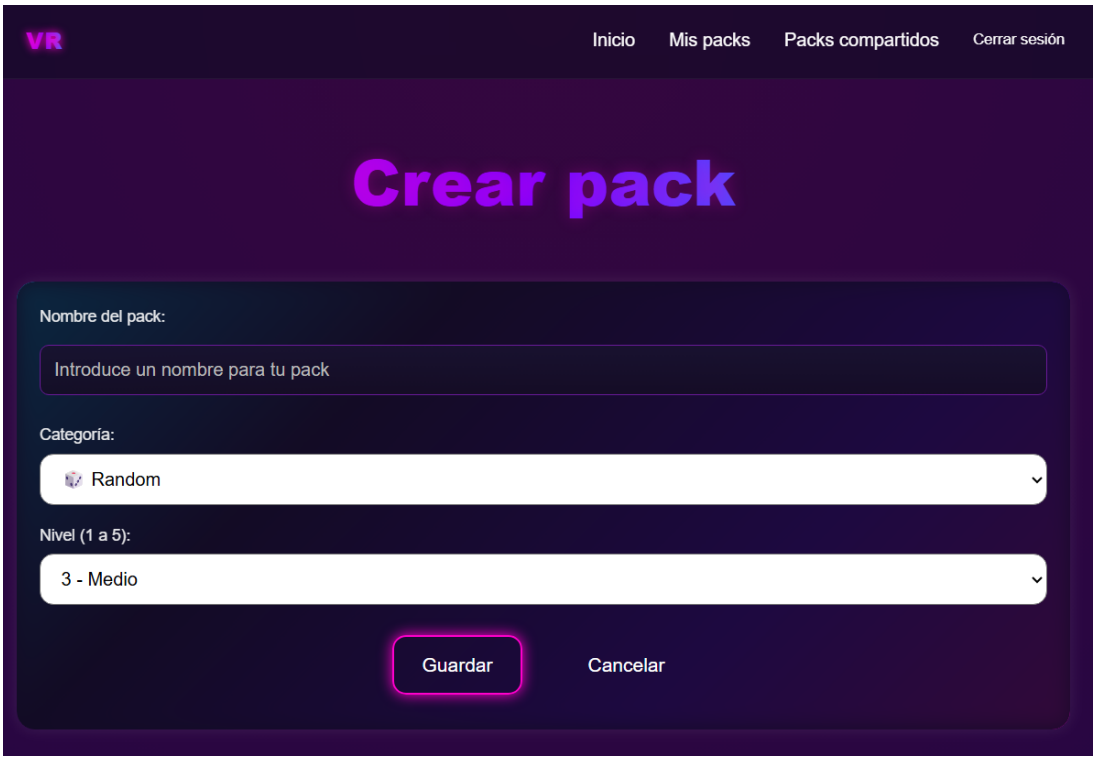

Figura A.6: Formulario de creación de un nuevo *pack*.

El usuario debe introducir un nombre identificativo para el *pack*, seleccionar una categoría y establecer un nivel de dificultad comprendido entre 1 y 5. Estos parámetros permiten clasificar y organizar el contenido posteriormente.

Una vez completados los campos requeridos, al pulsar el botón **"Guardar"**, el sistema valida la información introducida y crea el nuevo *pack*, que pasa a mostrarse automáticamente en la sección **Mis packs**.

En caso de que algún campo obligatorio no esté correctamente cumplimentado, el sistema mostrará los mensajes de error correspondientes.

A.5. Edición y gestión de un pack

Al seleccionar la opción de edición desde la sección **Mis packs**, el sistema muestra la vista completa de gestión del *pack*, desde la cual es posible modificar su información, administrar su contenido, gestionar colaboradores y configurar las opciones de acceso y compartición.

A.5.1. Información general del pack

La parte superior de la vista muestra la información general del *pack* seleccionado, tal como se observa en la Figura A.7.

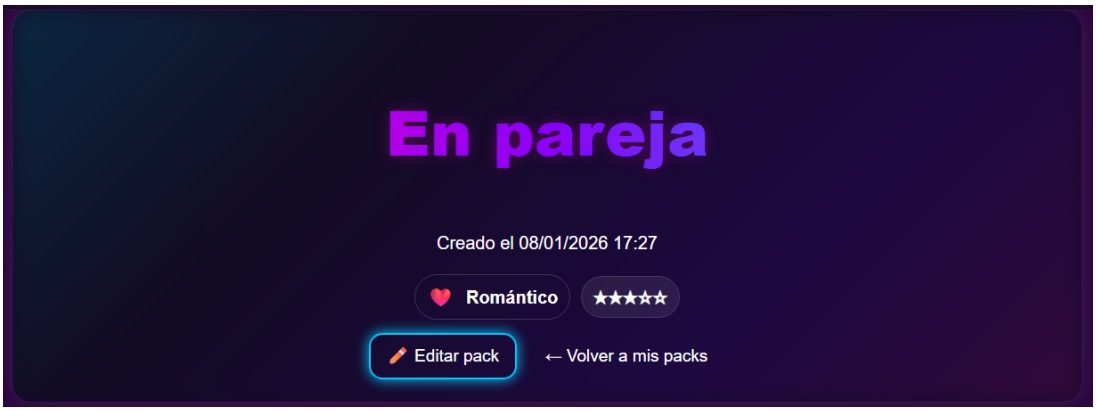

Figura A.7: Información general del pack.

En esta sección se muestran el nombre del *pack*, la fecha de creación, la categoría asociada y el nivel de dificultad. Estos parámetros pueden modificarse en cualquier momento mediante el botón **"Editar pack"**, que abre el formulario correspondiente para actualizar la información.

A.5.2. Colaboradores

Esta sección permite gestionar los usuarios que tienen acceso al *pack* y definir los permisos asociados a cada uno de ellos. La interfaz correspondiente se muestra en la Figura A.8.

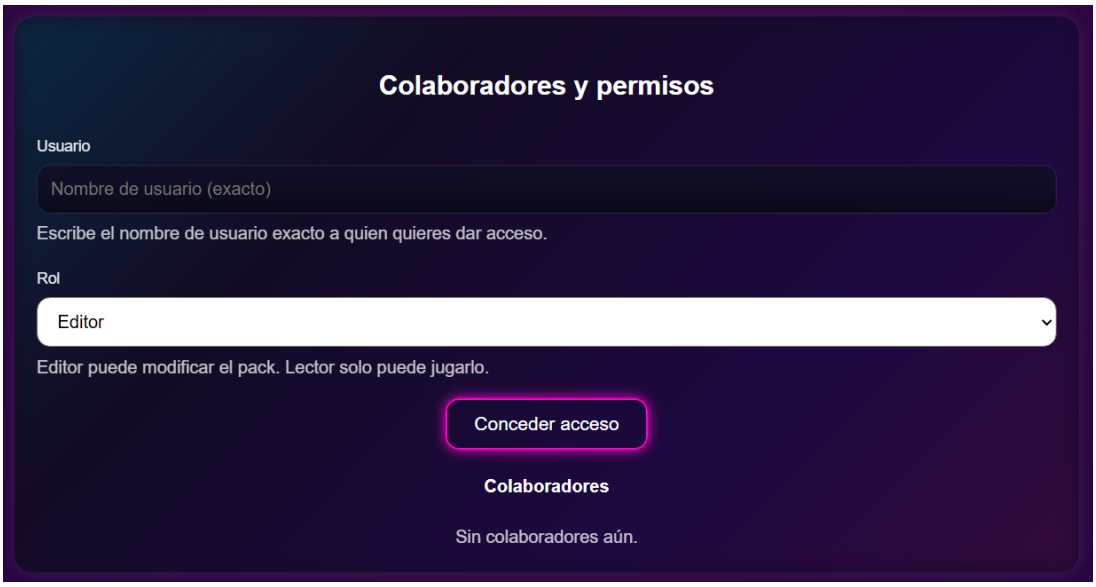

Figura A.8: Gestión de colaboradores y permisos del pack.

Para conceder acceso a otro usuario registrado en el sistema, es necesario introducir su nombre de usuario exacto en el campo correspondiente. A continuación, se debe seleccionar el rol que se desea asignar.

El sistema distingue entre dos tipos de roles:

- **Editor:** puede modificar el contenido del *pack*, incluyendo la creación, edición y eliminación de preguntas y retos.

- **Lector:** puede acceder al *pack* y jugarlo, pero no dispone de permisos para modificar su contenido.

Una vez definidos el usuario y el rol, al pulsar el botón **"Conceder acceso"** se registra el nuevo colaborador con los permisos seleccionados. En la parte inferior de la sección se muestra el listado actualizado de colaboradores asociados al *pack*.

Este mecanismo permite la edición colaborativa del contenido, facilitando la creación conjunta de preguntas y retos sin comprometer el control del propietario original del *pack*.

A.5.3. Gestión de preguntas y retos

La sección **Acciones del pack** permite administrar el contenido asociado al *pack*, diferenciando entre preguntas de tipo **Verdad** y desafíos de tipo **Reto**. La interfaz correspondiente se muestra en la Figura A.9.

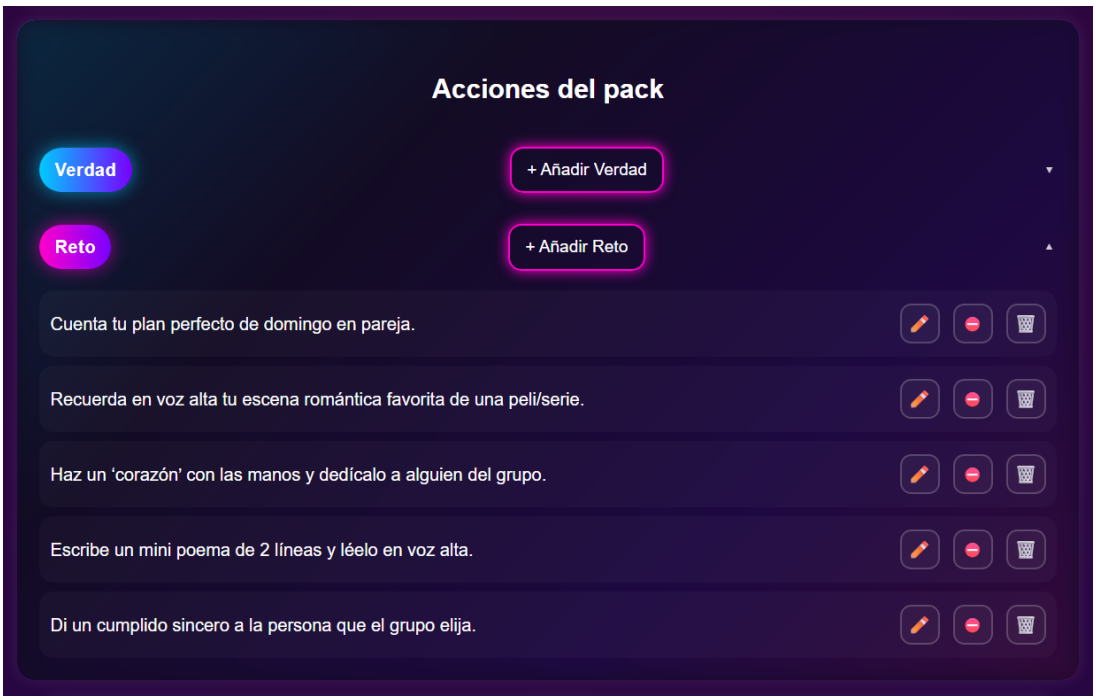

Figura A.9: Gestión de preguntas y retos del pack.

Desde esta sección es posible añadir nuevos elementos mediante los botones **"+ Añadir Verdad"** y **"+ Añadir Reto"**, que abren el formulario correspondiente para introducir el texto de la pregunta o desafío.

Cada elemento existente se muestra en una lista individualizada, junto con un conjunto de acciones disponibles:

- **Edición:** permite modificar el contenido del elemento seleccionado.
- **Activación o desactivación:** posibilita habilitar o deshabilitar temporalmente un elemento sin necesidad de eliminarlo.
- **Eliminación:** elimina de forma permanente la pregunta o reto del *pack*.

Este sistema permite mantener un control preciso sobre el contenido activo en cada momento, facilitando la actualización dinámica del *pack* y la adaptación del juego a distintos contextos o públicos.

A.5.4. Compartir y jugar

La última sección, **Compartir y jugar**, reúne las opciones necesarias para acceder al *pack* desde su vista pública y facilitar su distribución a otros usuarios. La interfaz correspondiente se muestra en la Figura A.10.

Figura A.10: Opciones de acceso, compartición y juego del pack.

En primer lugar, el sistema permite copiar el enlace público del *pack*, que puede compartirse directamente con otros usuarios. Dicho enlace conduce a la vista pública desde la cual puede iniciarse el juego.

Las acciones disponibles en esta sección son las siguientes:

- **Jugar:** abre la vista pública del *pack* y permite iniciar el juego en modo local.
- **QR:** genera y descarga un código QR que contiene el enlace público del *pack*, facilitando su acceso desde dispositivos móviles.
- **Regenerar:** crea un nuevo enlace público, invalidando el anterior. Esta opción resulta útil en caso de querer restringir accesos previamente compartidos.
- **Sala Online:** permite crear una sala remota asociada al *pack*, posibilitando la participación simultánea de varios usuarios desde distintos dispositivos.

Estas funcionalidades integran los mecanismos de distribución y acceso al contenido, permitiendo tanto el uso local como el juego en modalidad remota.

A.6. Vista pública del *pack*

La vista pública del *pack* es la página a la que acceden los usuarios al utilizar el enlace compartido, el código QR o el botón **"Jugar"** desde la sección de edición. Esta página está diseñada exclusivamente para la experiencia de juego, por lo que no muestra opciones de edición ni configuración. Su acceso no está limitado únicamente a usuarios registrados, sino que puede realizarse sin necesidad de autenticación.

La Figura A.11 muestra un ejemplo de esta interfaz.

Figura A.11: Vista pública del *pack*.

En la parte central se muestra de forma destacada una pregunta o reto perteneciente al *pack*. El sistema selecciona de forma aleatoria los elementos activos disponibles.

El usuario puede elegir entre las opciones **Verdad** y **Reto**, lo que determina el tipo de elemento que se mostrará a continuación. Tras cada interacción, el sistema presenta un nuevo contenido del mismo tipo seleccionado, permitiendo continuar la partida de forma secuencial.

A.7. Salas *online*

Las **Salas online** permiten jugar a un *pack* de manera remota con varios participantes conectados simultáneamente desde distintos dispositivos. Esta modalidad está orientada a facilitar partidas a distancia, manteniendo la sincronización del contenido mostrado a todos los usuarios.

La Figura A.12 muestra un ejemplo de una sala activa.

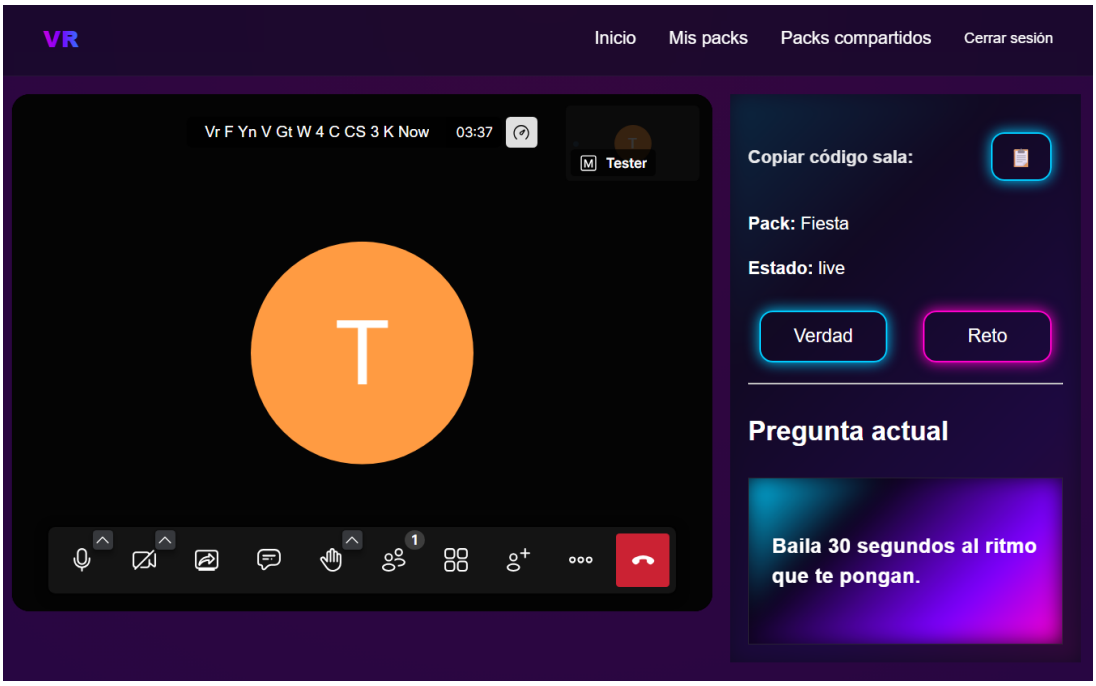

Figura A.12: Interfaz de una sala online con videocomunicación integrada.

Al crear una sala desde la sección **Compartir y jugar**, el sistema genera una sala asociada al *pack* y le asigna un identificador único. Este identificador forma parte de la URL de la sala, que puede copiarse para compartirla con el resto de participantes.

Una vez dentro, los usuarios visualizan el estado actual de la sala y el *pack* asociado. La sala se crea inicialmente en estado de espera (*lobby*) y es el *host* (creador de la sala) quien da comienzo al juego a través del botón correspondiente, cambiando el estado a *live*. Las acciones (Verdad o Reto) son igualmente controladas por el anfitrión y se sincronizan automáticamente para todos los participantes.

La interfaz integra, además, un sistema de videocomunicación que permite a los jugadores interactuar durante la partida, reproduciendo la dinámica del juego presencial en un entorno remoto.

A.8. *Packs* compartidos

La Figura A.13 presenta un ejemplo de esta vista.

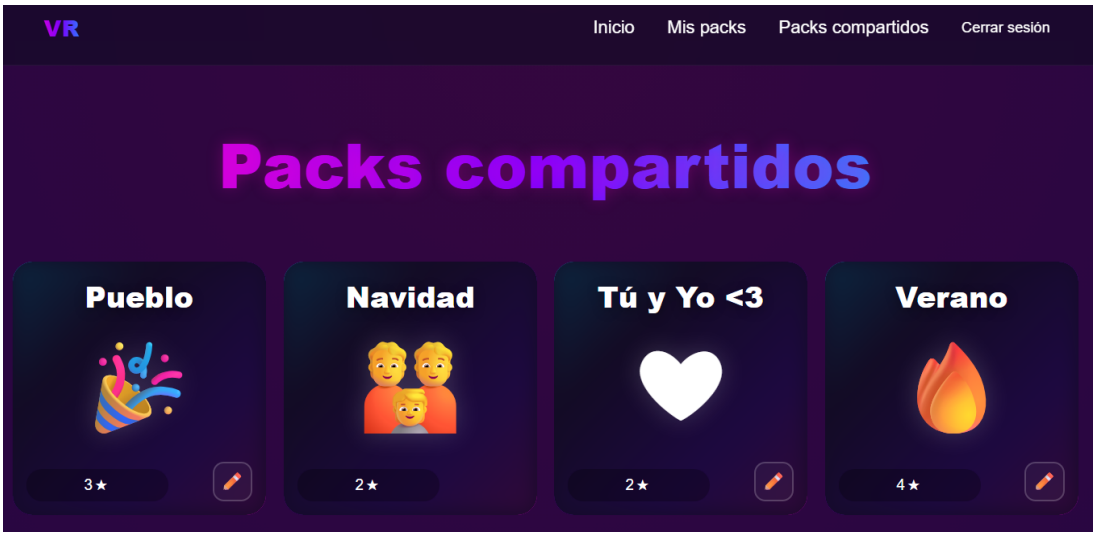

Figura A.13: Listado de packs compartidos con el usuario.

En esta página se muestran las tarjetas correspondientes a los *packs* compartidos, indicando su nombre y nivel de dificultad. Dependiendo del rol asignado (Editor o Lector), el usuario podrá modificar el contenido del *pack* o únicamente acceder a su vista pública para jugar.

Esta sección permite centralizar el acceso al contenido colaborativo, facilitando el trabajo conjunto entre usuarios y el uso compartido de los distintos *packs*.